

Chapter 6 I/O 接口 & 8255A

Dr. Prof. Ni Li

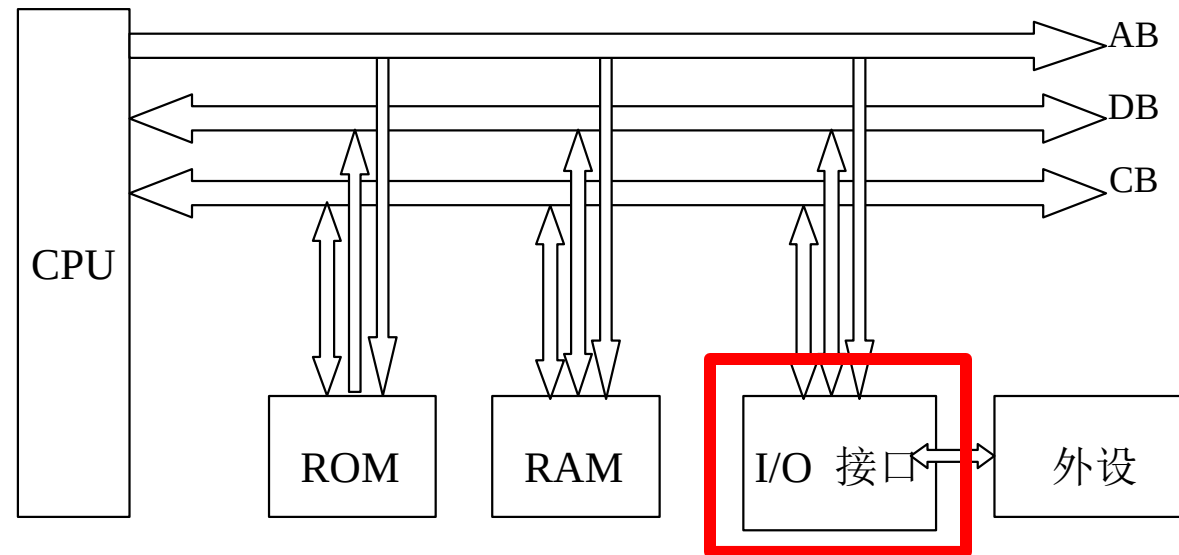
I/O 接口

I/O接口

- 微机组成: CPU, memory, I/O interface, system bus
- CPU基本组成, 管脚定义, 最小工作方式, 总线周期, 寄存器
- 软件: 汇编语言编程
- 后续关键: hardware interface硬件接口, CPU和外设之间的数据交换
 - IN/OUT

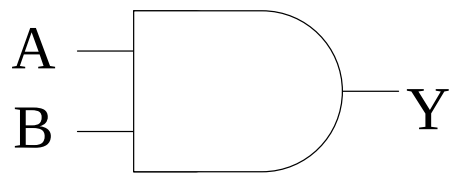
I/O interface

- 外设
 - Keyboard, monitor, printer, etc.
- I/O 接口
 - 连接外设与微机，协同工作，进行数据交换

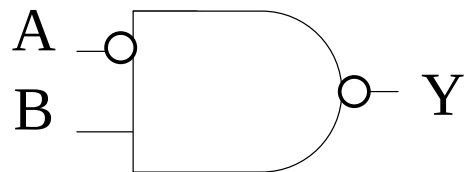


- 进一步理解IO接口的作用，如何基于IO接口进行数据交换

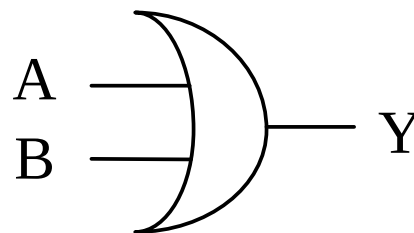
电路符号



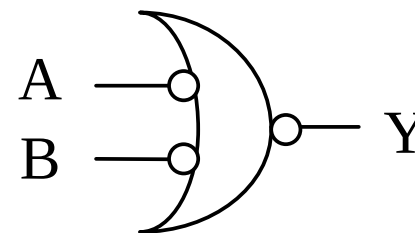
A=1
and B=1
then Y=1



A=0
and B=1
then Y=0

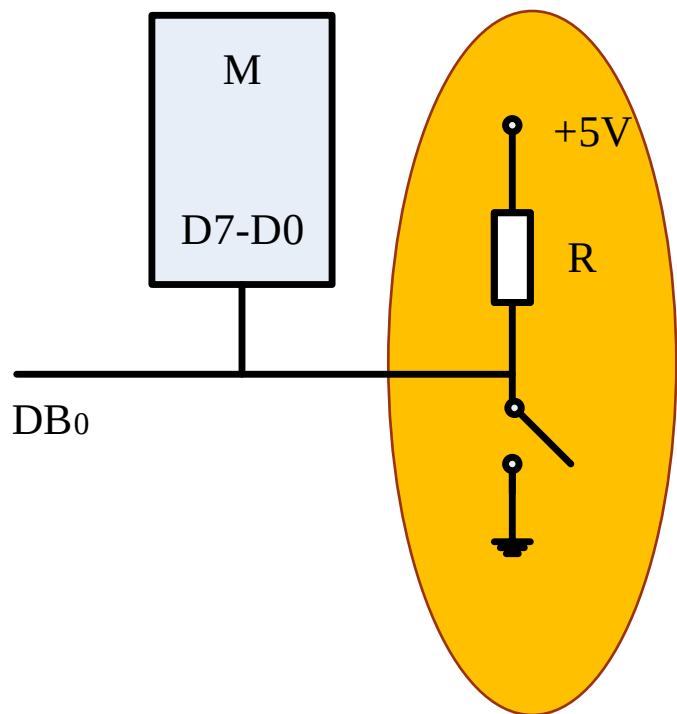


A=1
or B=1
then Y=1



A=0
or B=0
then Y=0

I/O接口功能

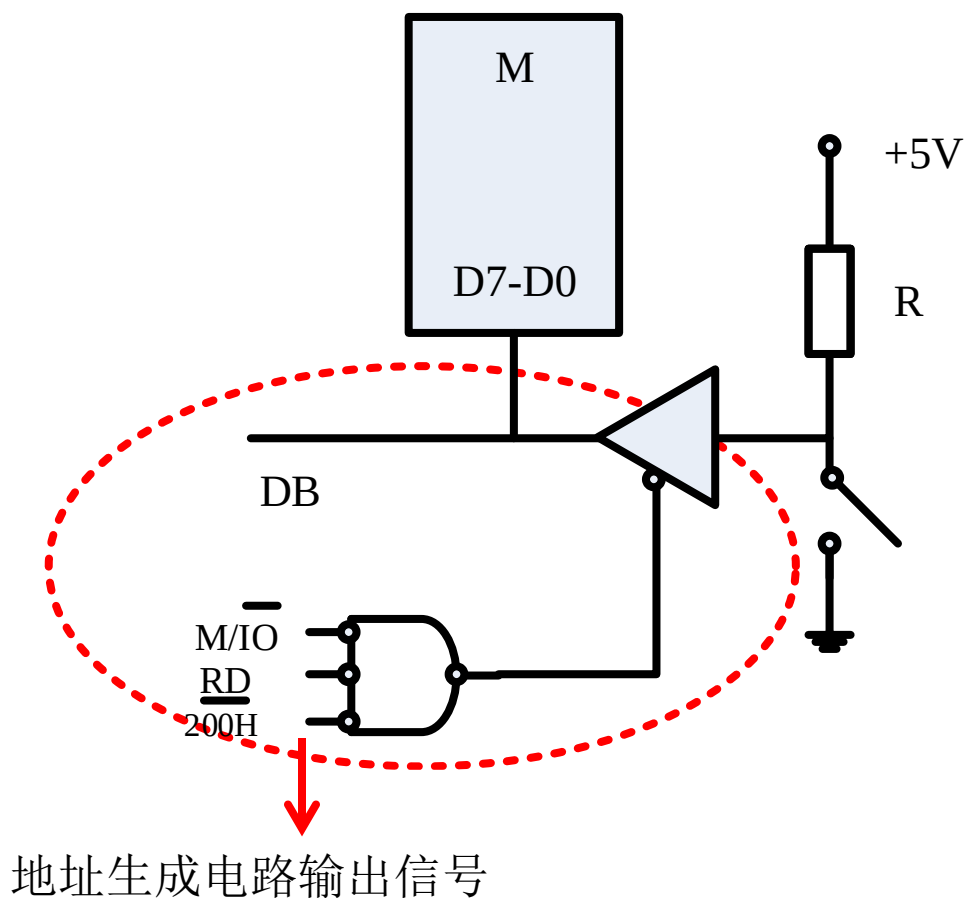


- 输入设备
 - 读取开关状态
- 问题:当设备直接连接到数据总线
 - DB: 干扰数据总线信号
 - AB: 不能区分外设(没有对外设分配地址)
 - CB: 没体现控制信号作用
 - $\overline{M/IO}$, $\overline{RD}$

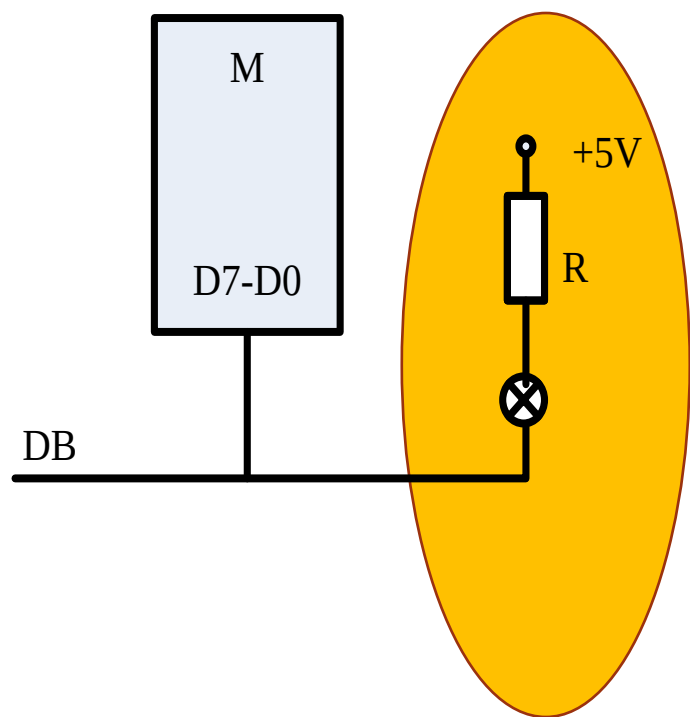
I/O接口功能

- **I/O 接口**: 带片选信号的缓冲器
 - 三态
- **输入指令**

地址分配?
控制信号作用?
干扰数据总线?



I/O接口功能

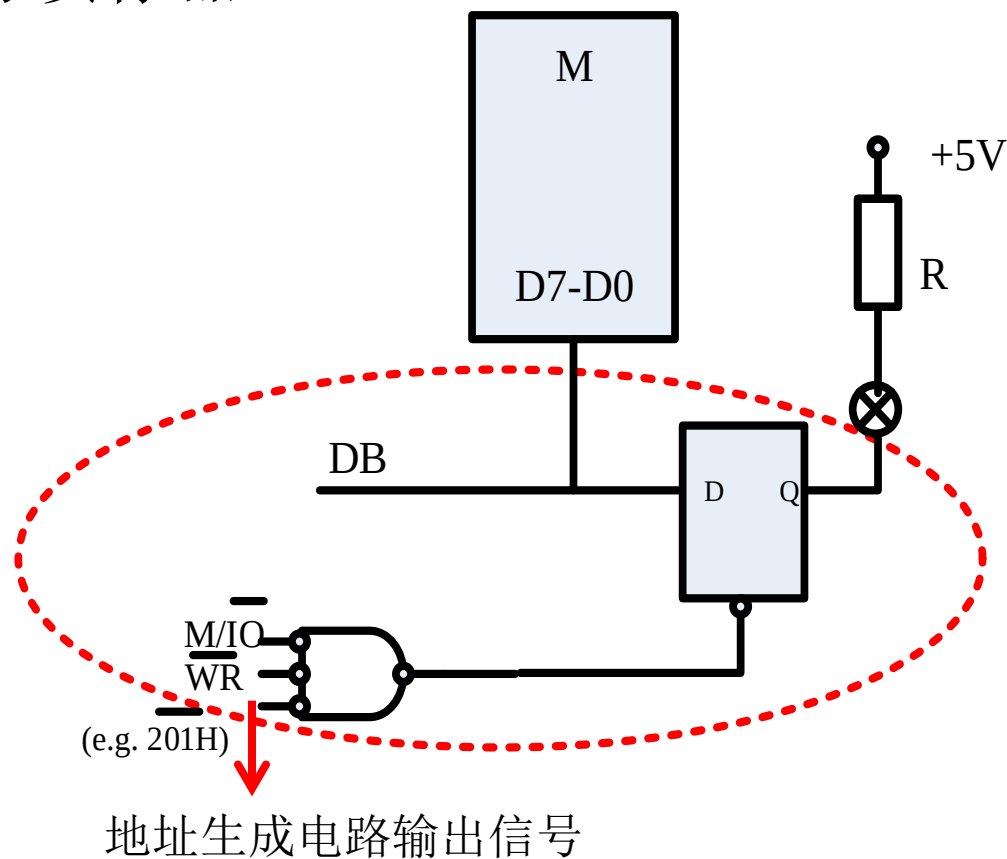


- Output devices 输出设备
 - LED灯
- 问题:当设备直接连接到数据总线
 - DB: 数据线上的输出信号不能保持
 - AB: 不能区分外设(没有对外设分配地址)
 - CB: 没体现控制信号作用
 - M/IO, WR

I/O接口功能

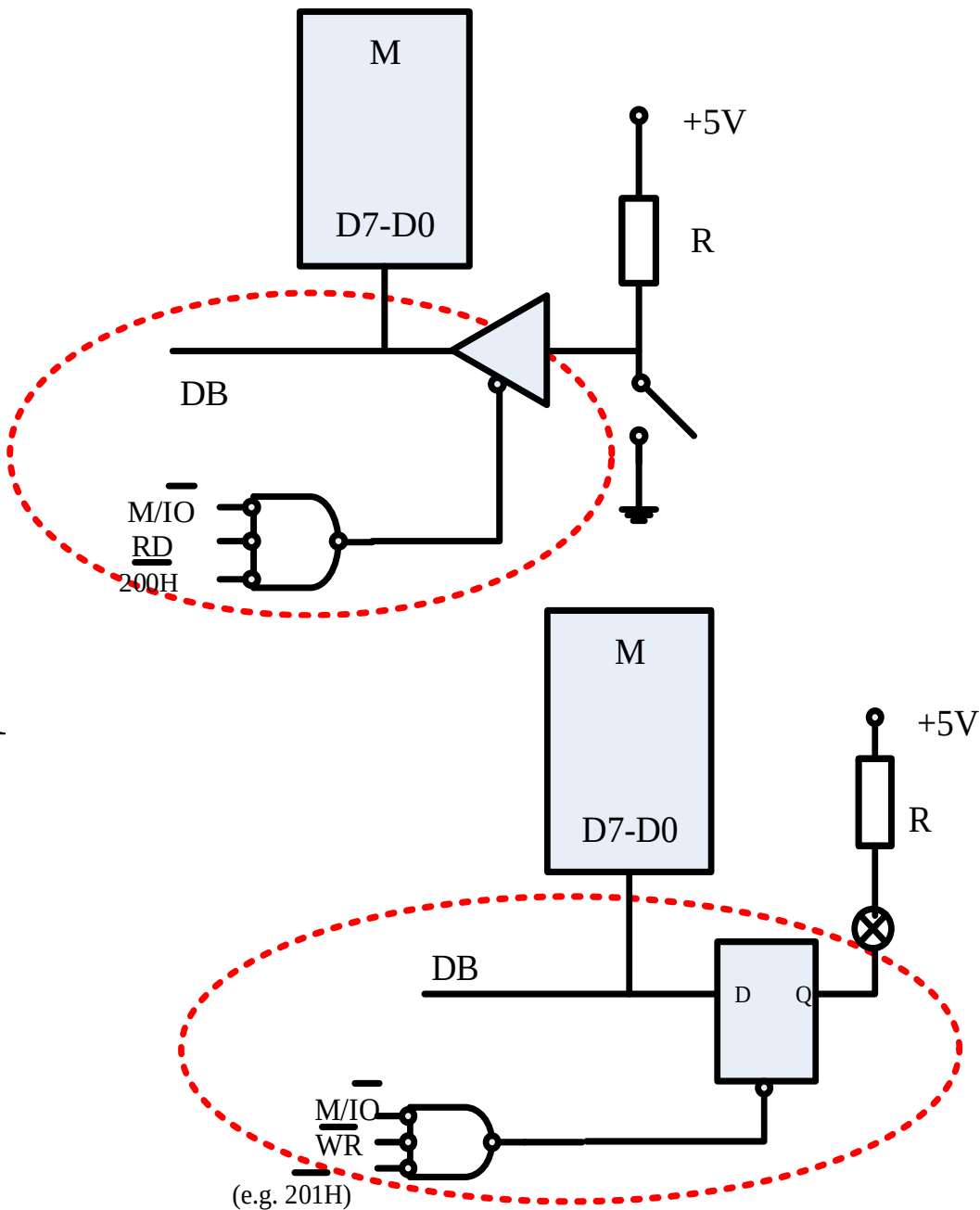
- **I/O interface:** 带片选信号的锁存器
- 输出指令

输出信号可以锁存？
地址分配？
控制信号作用？



I/O接口功能

- 输入缓冲
- 输出锁存
- 外设编址使用
- 电压转换
- 信息格式转换: A/D, D/A
- 时序控制: 同步
- 可编程控制



I/O 信息组成

- 数据信息(必需)
 - E.g. : 输入数据(keyboard); 输出数据(monitor)
- 状态信息
 - 外设工作状态信息
- 控制信息
 - 控制外设工作方式

数据信息

- 数字信息
 - 二进制: byte word
 - E.g. : CPU 与键盘, 监视器, 打印机之间的数据交换
- 模拟信息
 - 控制系统
 - 传感器: 非电信号->电信号
 - 温度、压力、流量、位移等等 etc. -> 电压、电流
 - AD/DA
- 开关信息
 - 两种状态: 开-关 (开关, 阀门 etc.)
 - 1个二进制位表示: 1/0

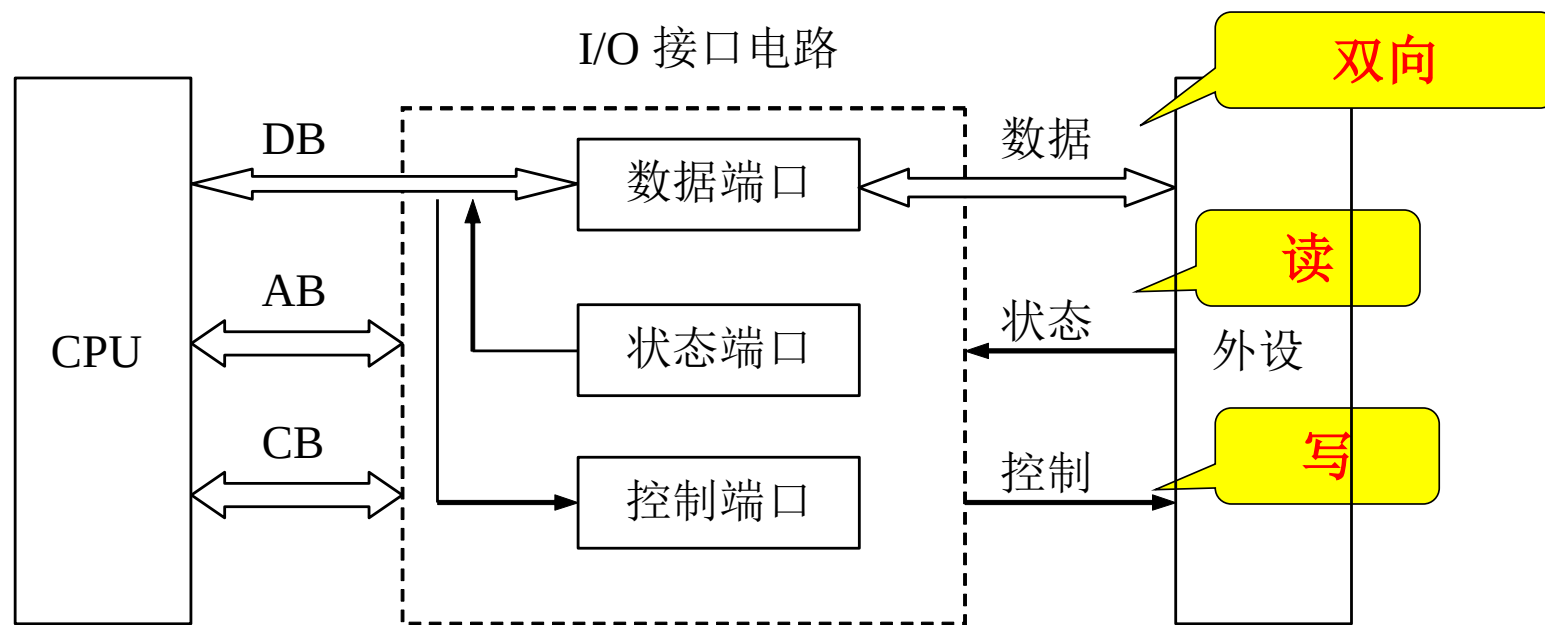
状态/控制信息

- 状态信息
 - CPU与外设间的联络信号
 - 通过读取状态信号,了解外设的工作状态
 - 常用状态位信息: Ready, Busy, Error
- 控制信息
 - 设置外设的工作方式(命令字)
 - 常用控制位信息: Start, stop, interrupt enabled etc.

I/O 端口

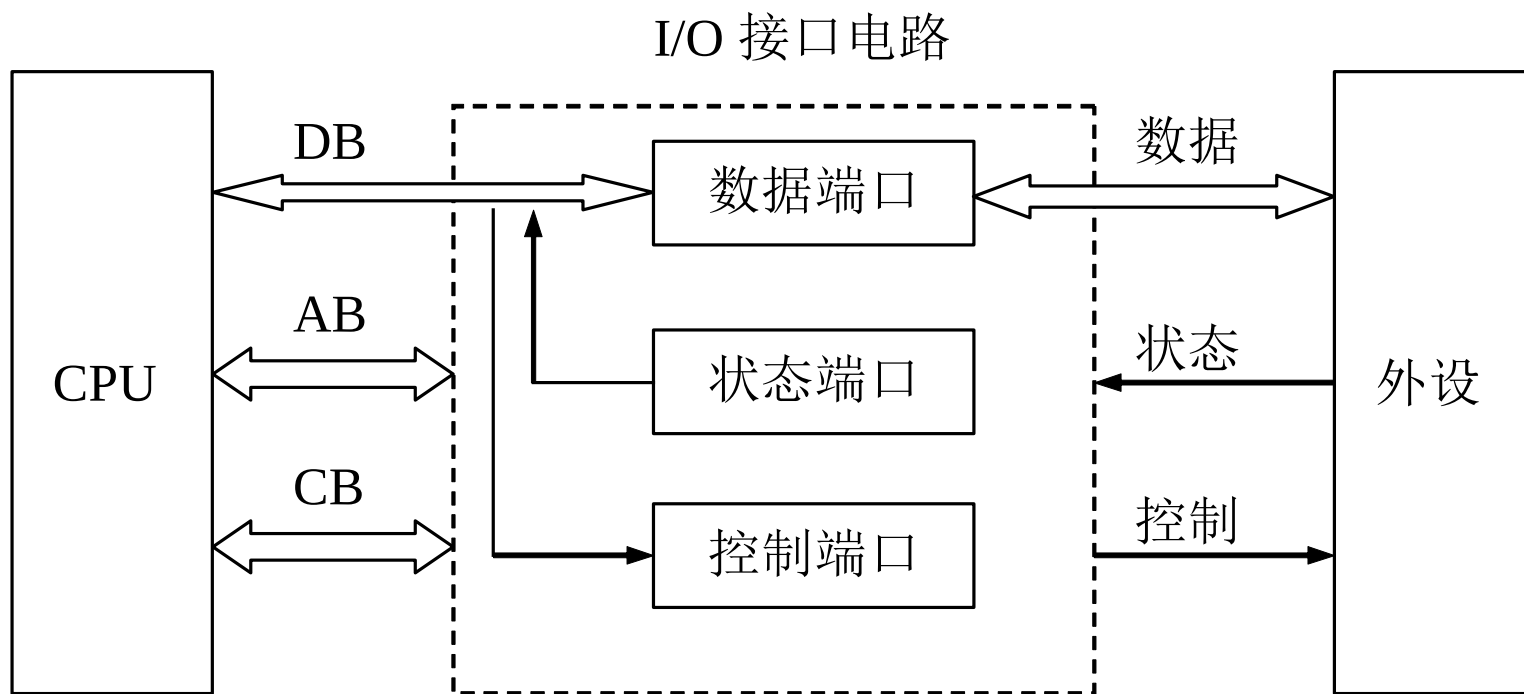
- I/O 端口

- 用于存储和交换不同信息的寄存器和控制逻辑 (control, state, data information)
- 地址区分：控制/命令端口, 状态端口, 数据端口



I/O端口

- 1个外设一般有多个端口(**不同地址**)
 - 数据口: 存储和交换数据信息
 - 状态口: 显示外设当前状态信息, CPU读取以检查当前状态 (TEST)
 - 控制口: 存储CPU的命令字或者控制字, 定义外设工作状态

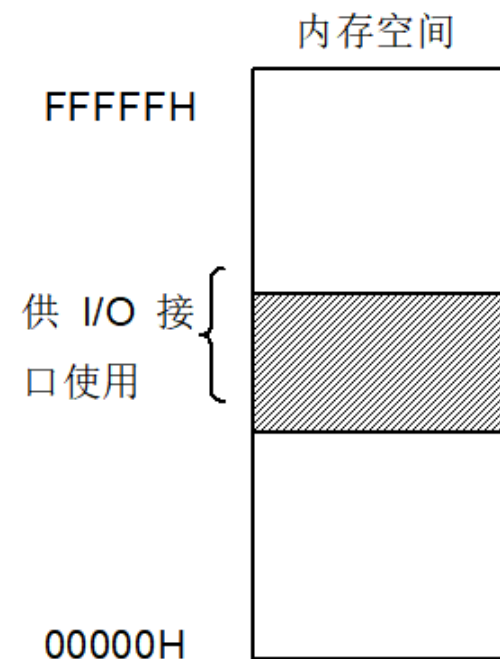


I/O 端口寻址

- 多个外设→多个I/O 接口电路→多个 I/O端口
- I/O端口寻址?
 - 为每个端口分配地址(数据端口, 状态端口, 控制端口)
 - 类似对存储单元分配地址
- 两种寻址方式
 - 存储器映射方式(统一编址unified addressing for I/O ports)
 - I/O 独立编址(特殊 I/O 指令, 64K I/O 空间)

I/O 端口寻址

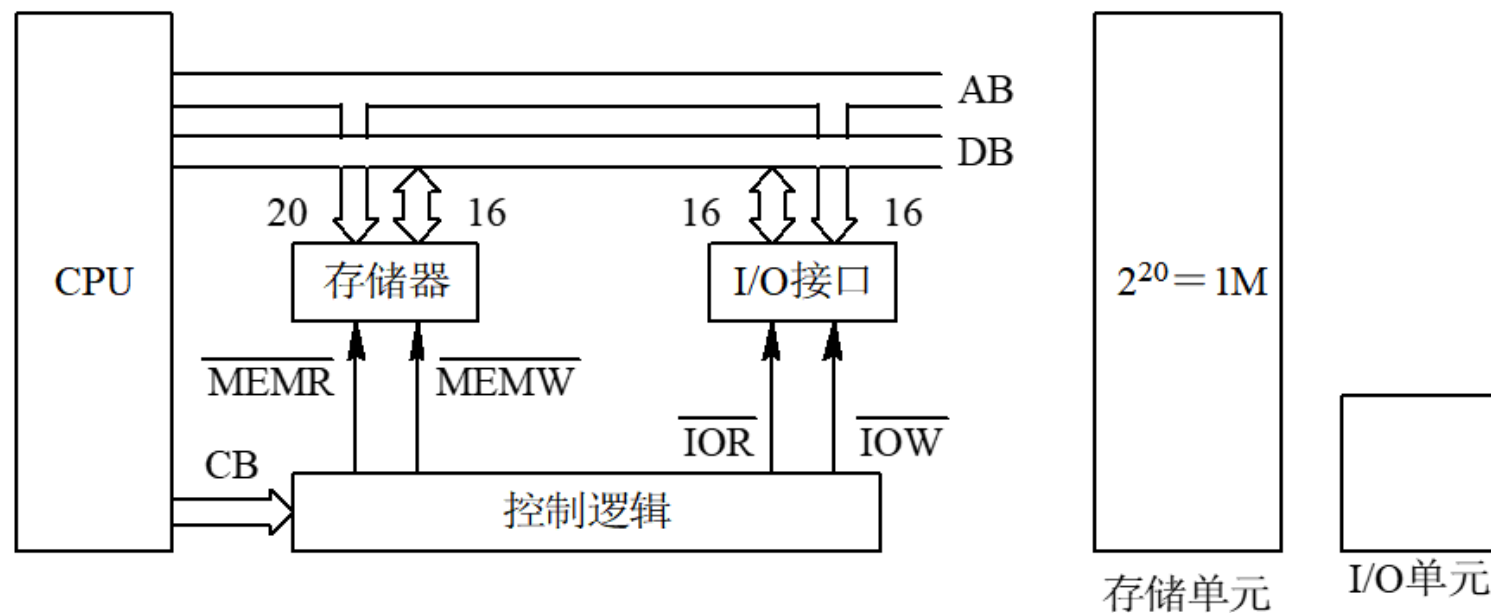
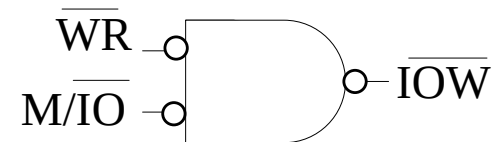
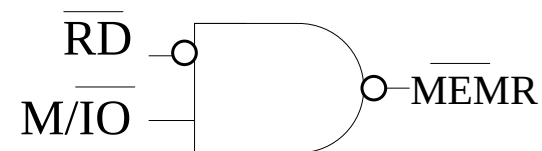
- 存储器映射方式(统一编址)
 - I/O 端口 视为存储单元
- **优点:** 没有特殊的IN/OUT指令，如同访问存储单元一样访问IO 端
- **缺点:** I/O 端口占用一定内存空间，需要更复杂的地址生成电路
 - Motorola MC6800, ARM



(a) 存储器映射方式示意图

I/O 端口寻址

- I/O 独立编址
- 两个独立的地址空间
 - 专门的I/O指令
 - 64K I/O 端口地址空间

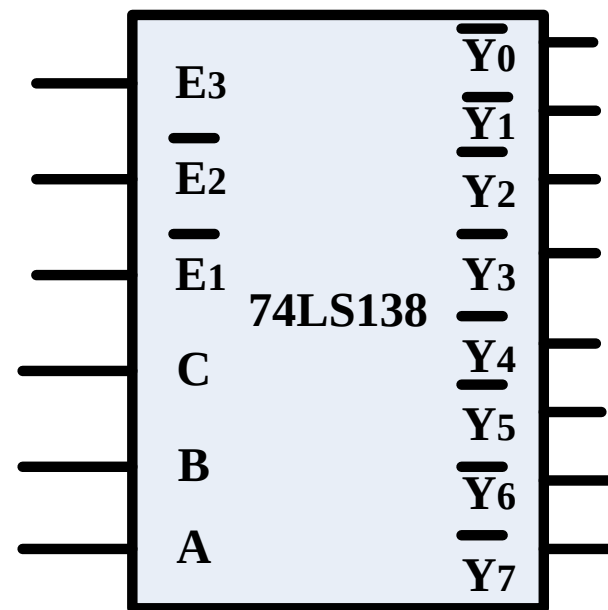


I/O 端口寻址

- I/O独立编址: 优点
 - 地址译码电路简单（短地址）
 - 程序中明确区分对M和IO的访问
 - 分离结构下，可以独立设计M和IO电路
- I/O独立编址: 缺点
 - 需要特殊的 I/O读写指令, 缺少存储器访问指令的灵活性
- 解决方法: 读入 I/O 信号到M→用内存访问指令处理数据→输出I/O信号

I/O端口地址生成

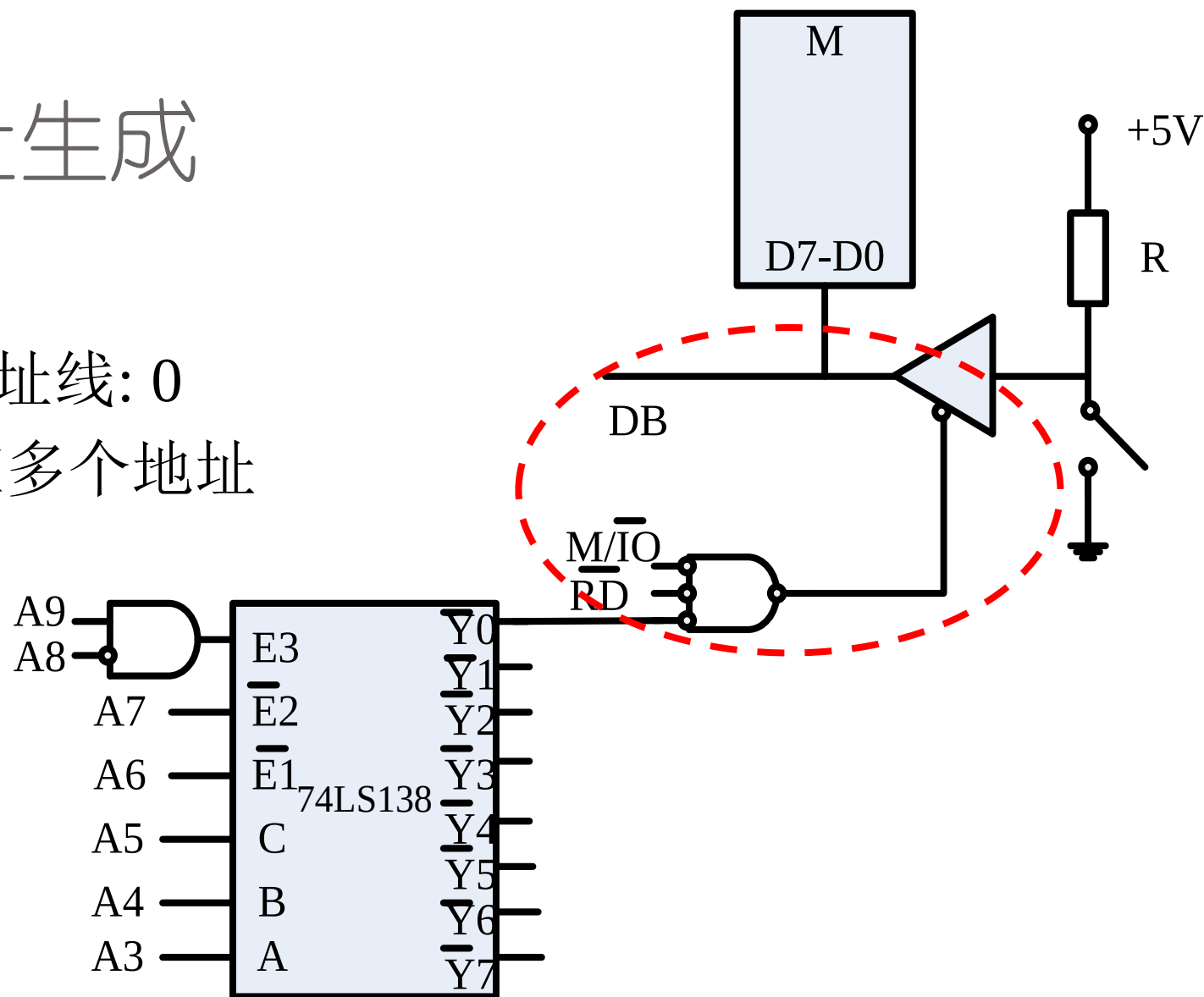
- 地址译码器: 74LS138
 - 外设的地址生成与地址译码
 - 真值表



E3	$\overline{E2}$	$\overline{E1}$	C	B	A	$\overline{Y0}$	$\overline{Y1}$	$\overline{Y2}$	$\overline{Y3}$	$\overline{Y4}$	$\overline{Y5}$	$\overline{Y6}$	$\overline{Y7}$
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
.....													

I/O端口地址生成

- 输入: 连接地址总线
 - 若未使用的高位地址线: 0
- 未使用地址线: 对应多个地址



8255A并行接口

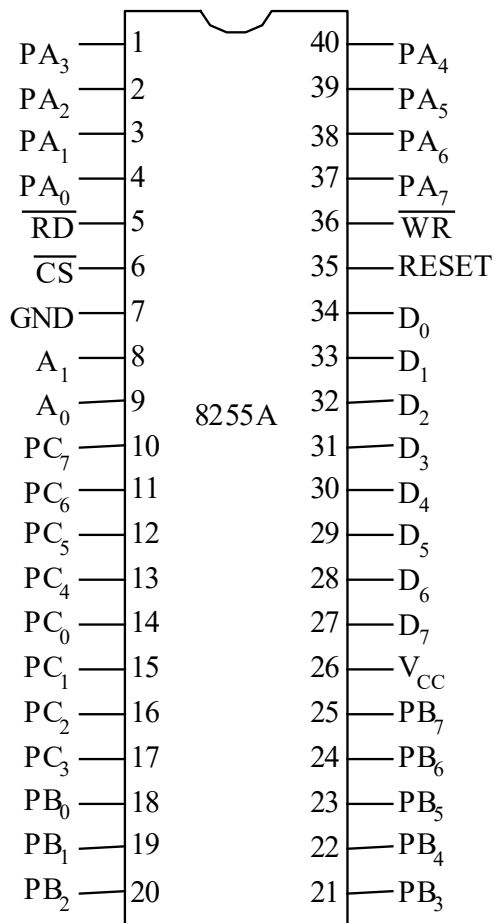
8255A

- 并行接口: CPU和外设间并行传输
- 串行接口: 沿一条数据线一位一位传输
- 8255A可编程 I/O 接口芯片: 并行数据传输接口

支持输入缓冲、
输出锁存

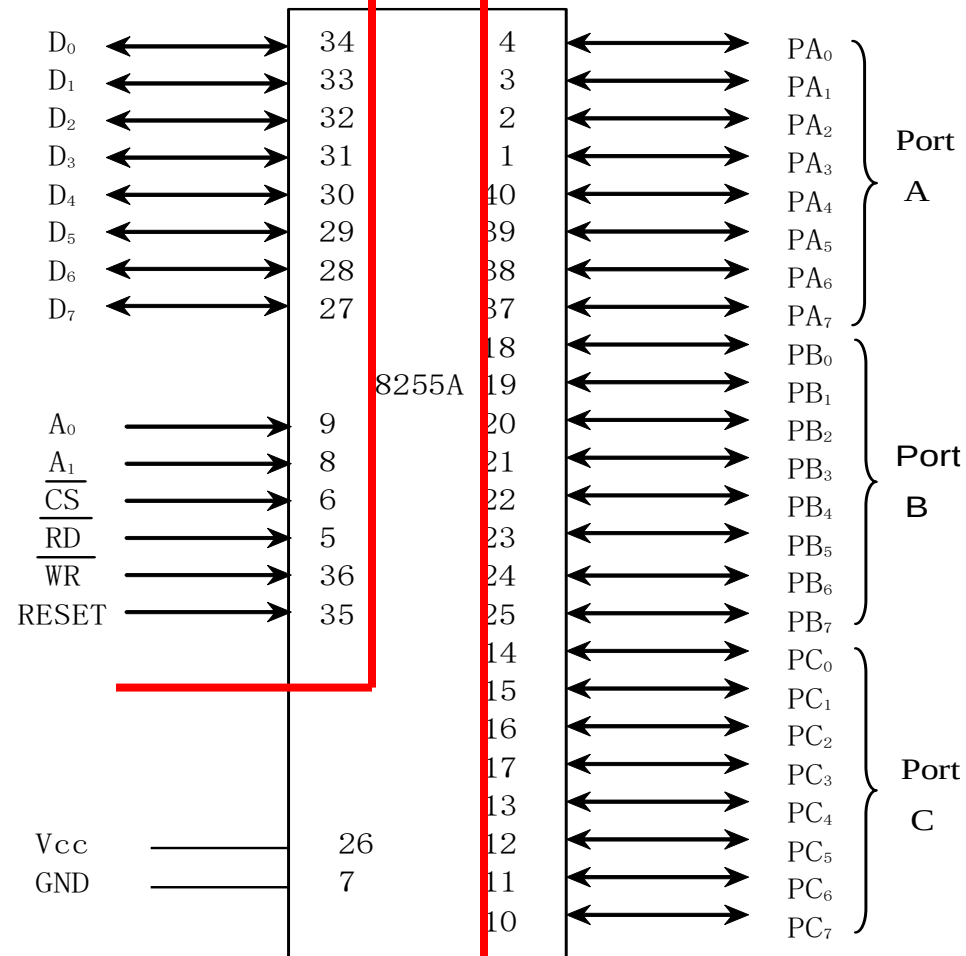
8255A管脚

3个8-bit 并口



连接系统总线

连接外设



A口 B口 C口(PC7-4 PC3-0)

A₁A₀分别选通: A口/B口/C口/控制字R $\overline{CS}$ $\overline{RD}$ $\overline{WR}$

8255A基本操作

A_1	A_0	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	操作
0	0	0	1	0	读操作
					A口到数据总线
					B口到数据总线
					C口到数据总线
1	0	0	1	0	写操作
					数据总线到A口
					数据总线到B口
					数据总线到C口
1	1	1	0	0	写控制字

8255A工作方式

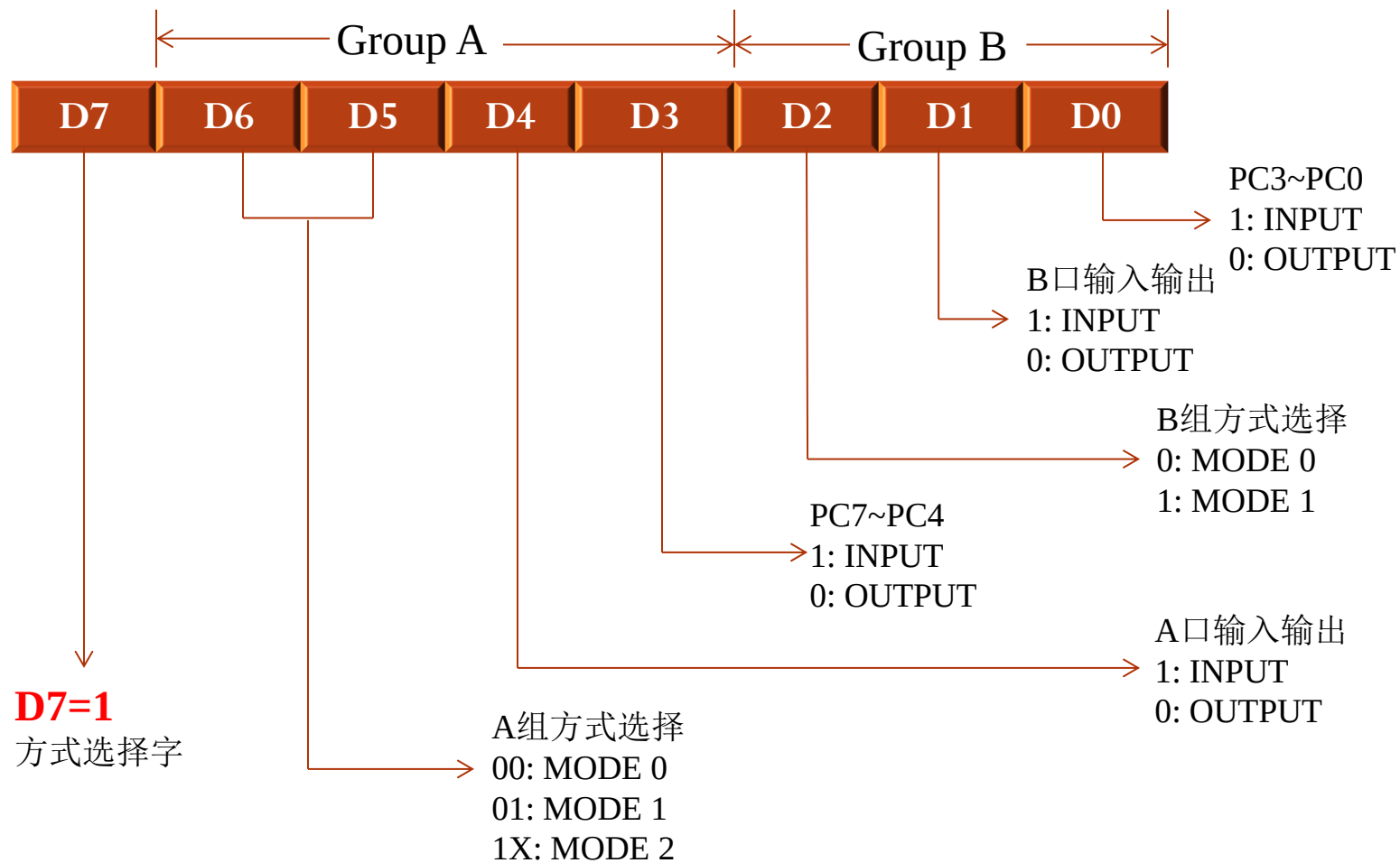
- 工作方式
 - **方式 0: 简单 Input/Output (直接输入/输出, 无联络信号)**
 - 方式 1: Input/Output (带握手信号)
 - 方式2: 双向 I/O 数据传输
- A口
 - 方式0,1,2 → 需2个二进制位进行设置
- B口
 - 方式0,1 → 需1个二进制位进行设置
- C口
 - 方式0

方式 0

- 直接输入/输出
- 读/写
 - 通过IN指令从指定端口读数据, $\overline{RD}=0$
 - 通过OUT指令向指定端口写数据, $\overline{WR}=0$
 - 无需联络信号

控制字

- 控制字：A、B、C口工作方式、输入或输出

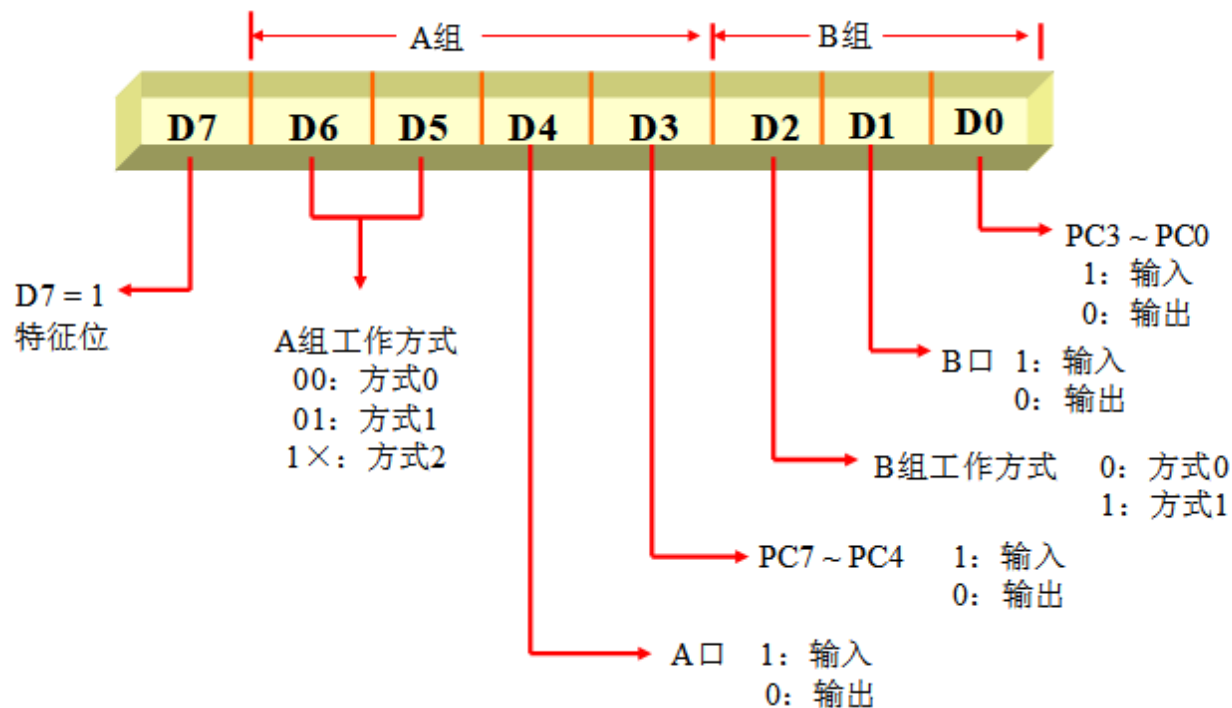


Example

- 8255A 地址300H~307H, 8255A的A1、A0 连接地址总线低2位

A \B \C \Control R: 端口地址多少?

- 初始化8255A, A口输入, B口输出, 方式0
- 初始化后, 从A口读入数据; 从数据段偏移地址为BX的单元取数并输出到B口



Example

MOV DX, 303H; 此时A1A0=11, 选中控制字寄存器

MOV AL, 10011001B;

OUT DX, AL; 首先设置8255工作方式

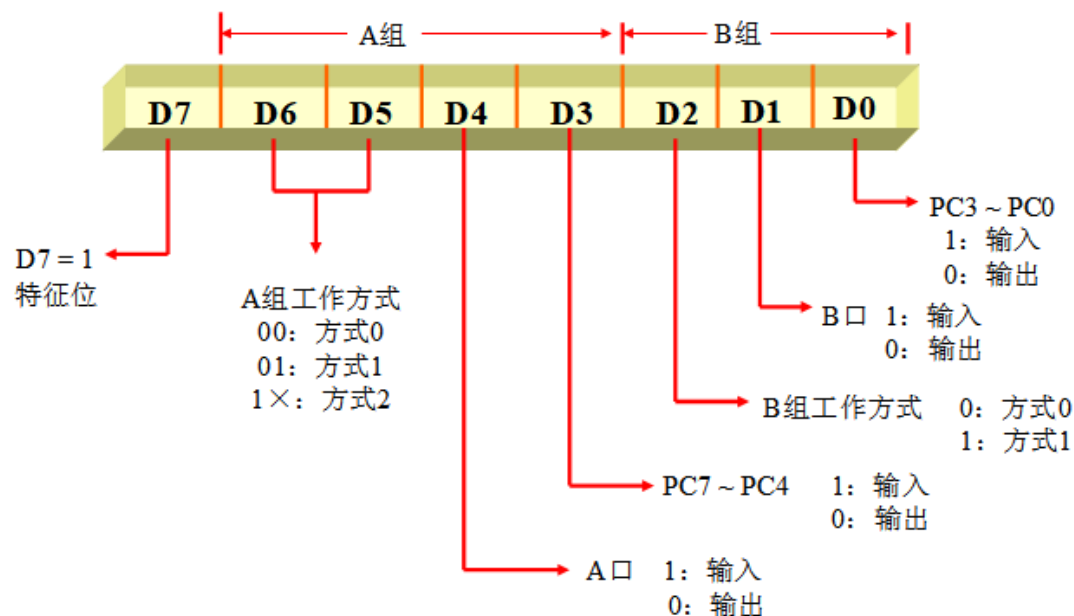
MOV DX, 300H; 此时A1A0=00, 选中A口

IN AL, DX; 从A口读入数据

MOV AL, [BX]; 从DS:BX的存储器中取数

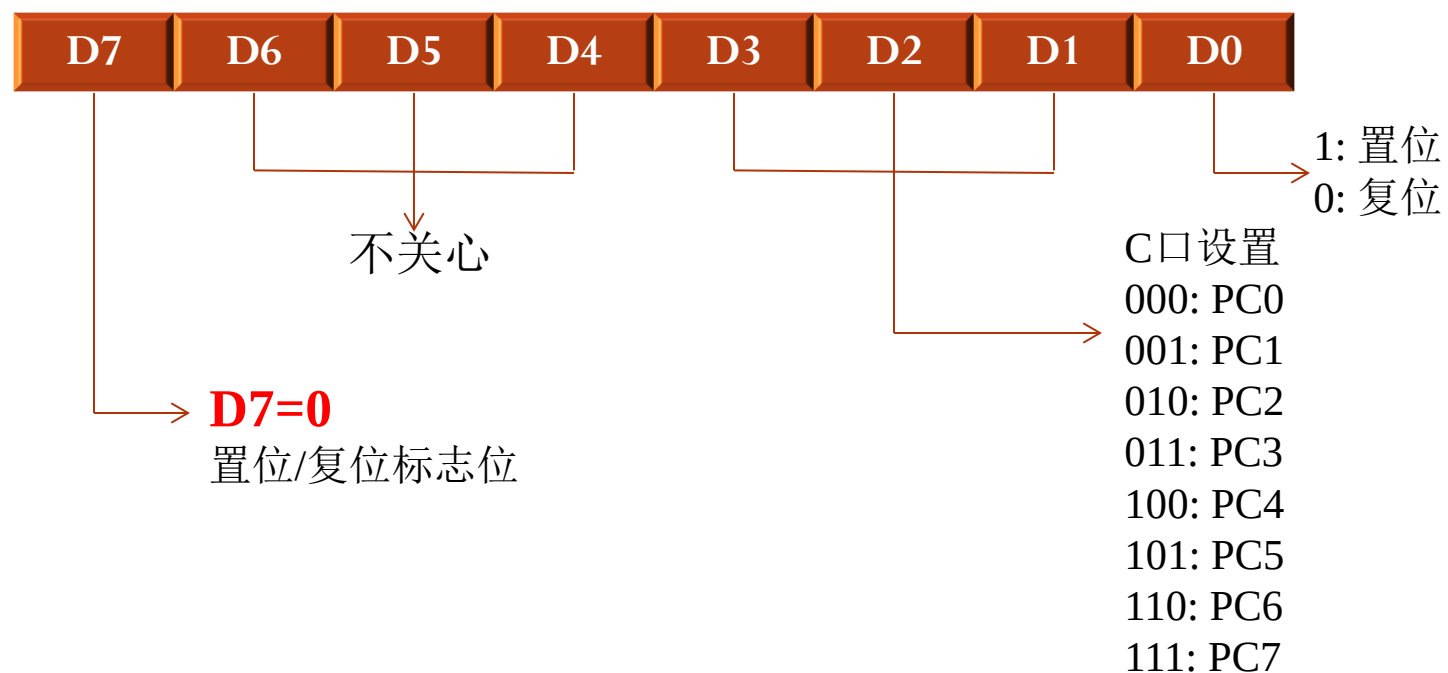
MOV DX, 301H; 此时A1A0=01, 选中B口

OUT DX, AL; 从B口输出数据



Control Word控制字

- 置位控制(Port C)



Example

- 8255A 地址60H~63H, 且8255A的 A1, A0连接地址总线低2位

端口地址?

- 从PC5输出正脉冲

- 输出低电平
- 输出高电平
- 输出低电平

```
MOV AL, 00001010B;
```

```
OUT 63H, AL;
```

```
CALL DELAY
```

```
MOV AL, 00001011B;
```

```
OUT 63H, AL;
```

```
CALL DELAY
```

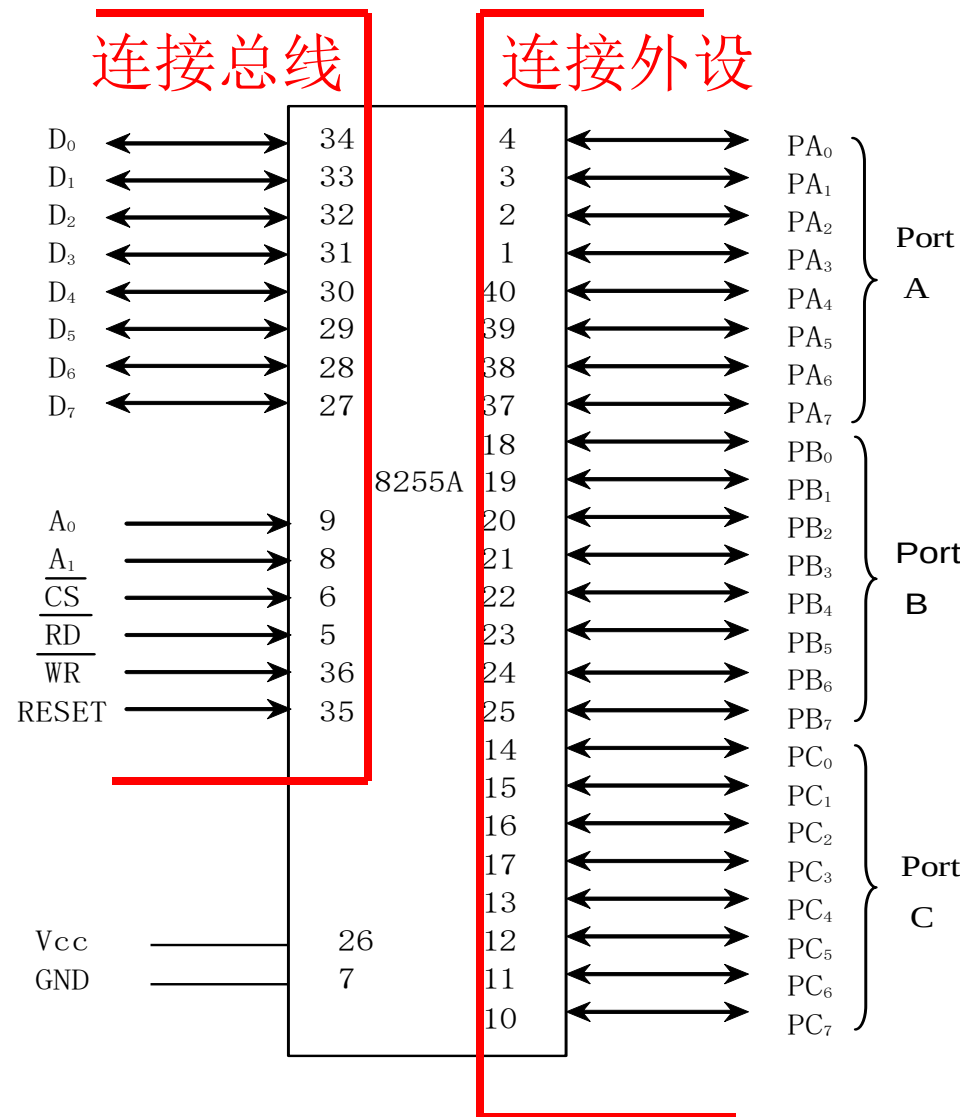
```
MOV AL, 00001010B;
```

```
OUT 63H, AL;
```

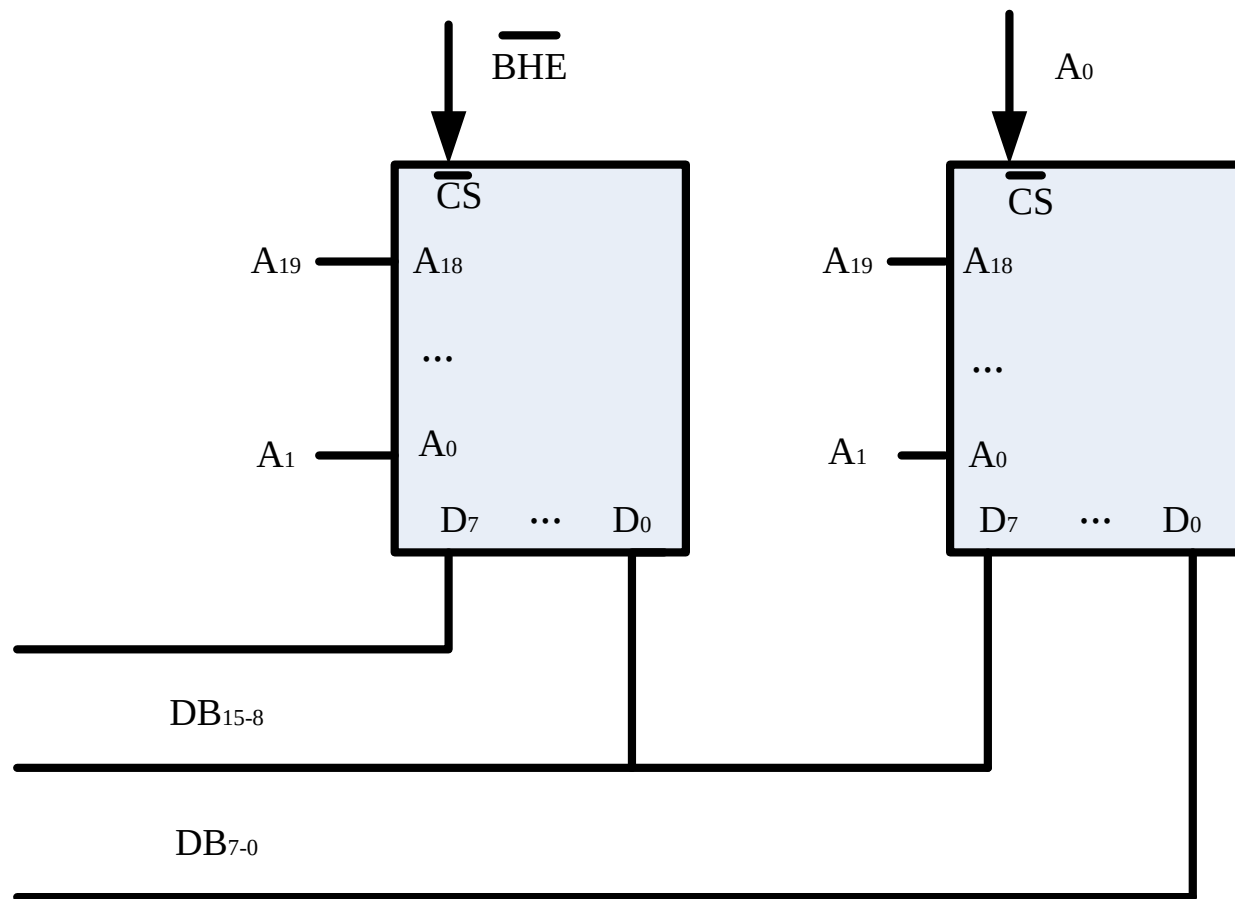
```
CALL DELAY
```


8255A和数据总线的连接

- 8255A 是8位接口芯片,
8086 CPU有 16 位数据线
- 2种连接方式



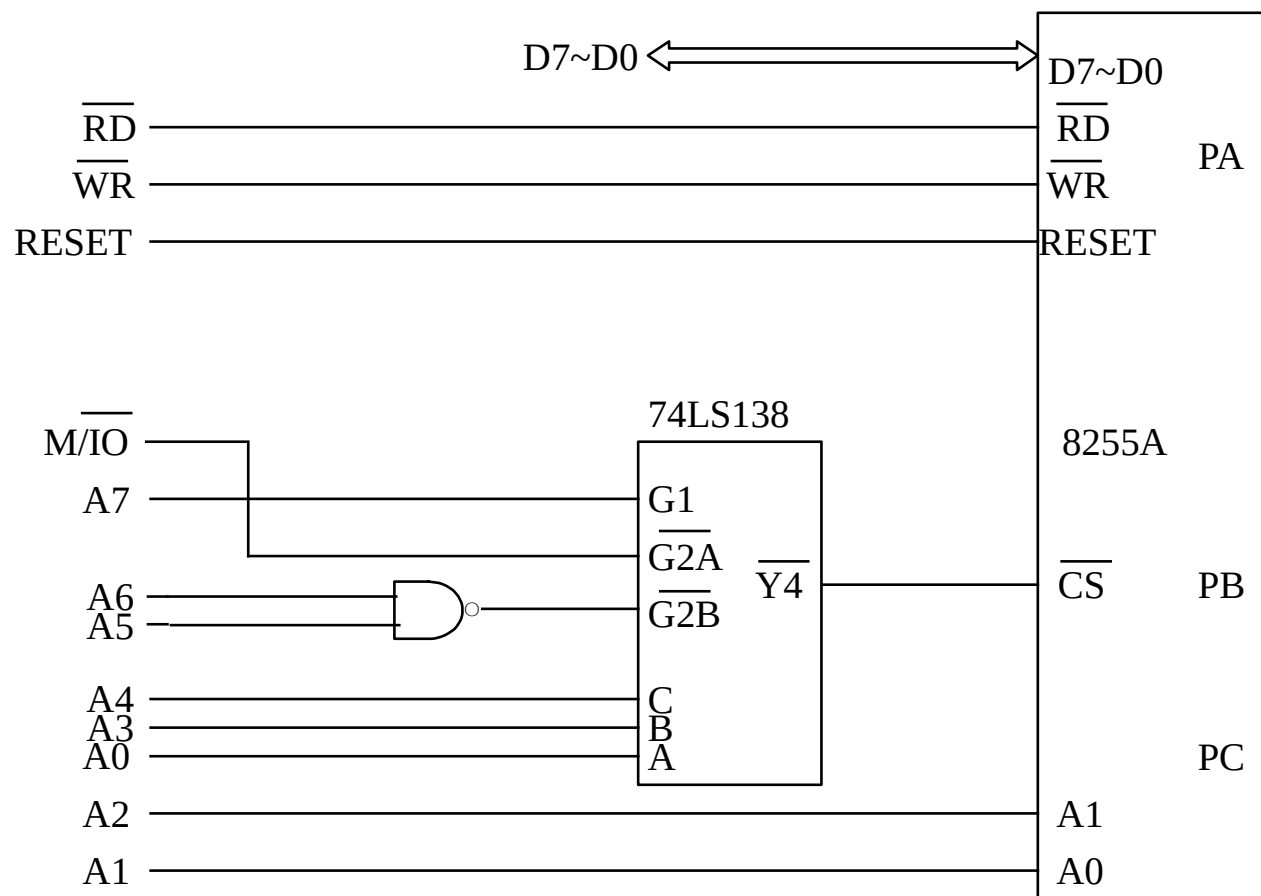
8086 内存结构



- 使用奇地址 ($\overline{\text{BHE}}=0$) $\rightarrow$ 高8位数据线有效
- 使用偶地址 ($A_0=0$) $\rightarrow$ 低8位数据线有效

1 简易接法

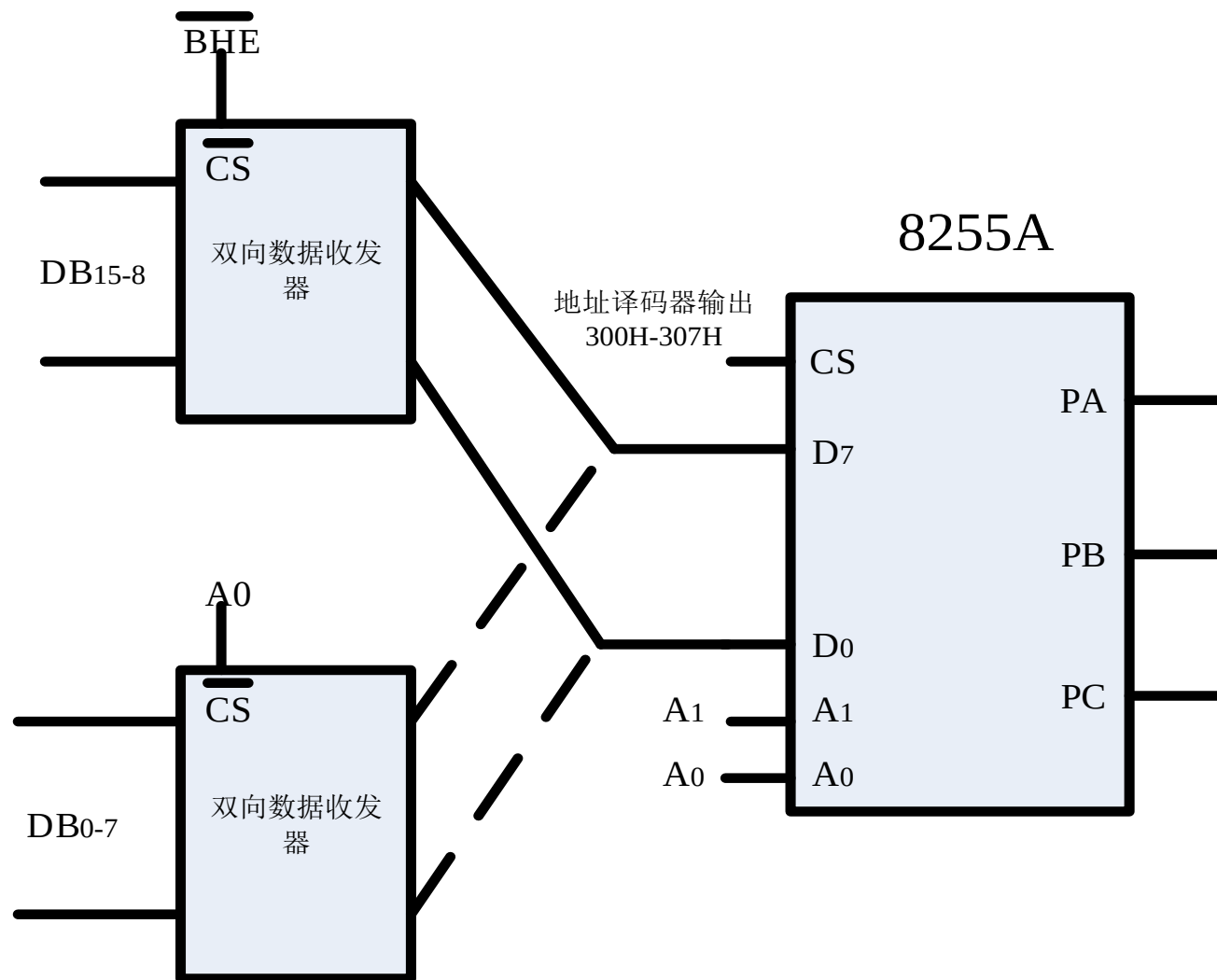
- 只使用数据总线的低（/高）8位
- 只使用偶(/奇)地址



F0H,
F2H,
F4H,
F6H

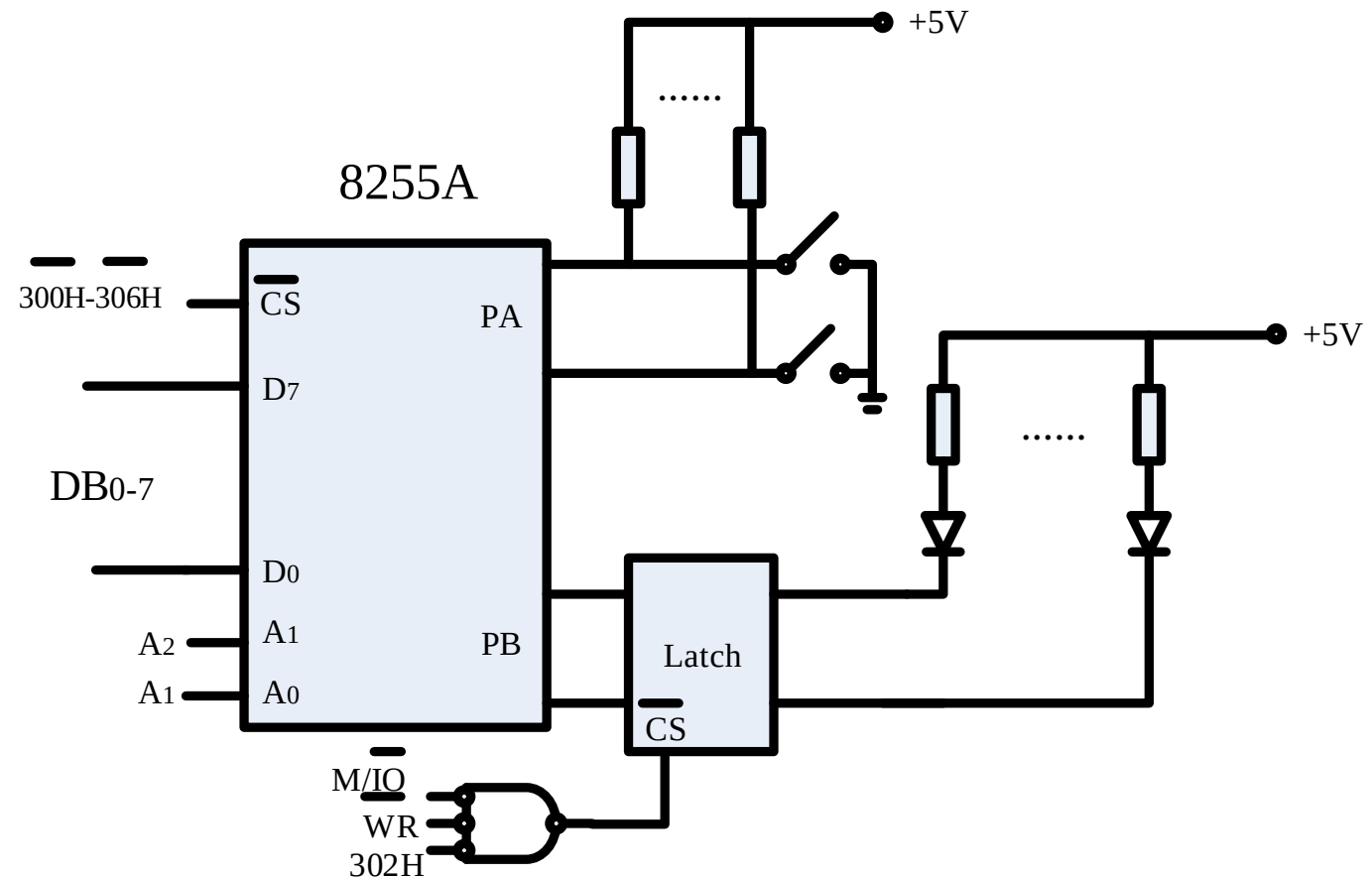
2 使用16位数据总线

- 连接到DB (16 bits): 双向数据收发器(片选信号)



Example

- 从A口读取端口数据, 从B口输出, 使对应LED灯亮。
按任意键退出程序
- 端口地址?



程序流程

● INT 21H Function 6

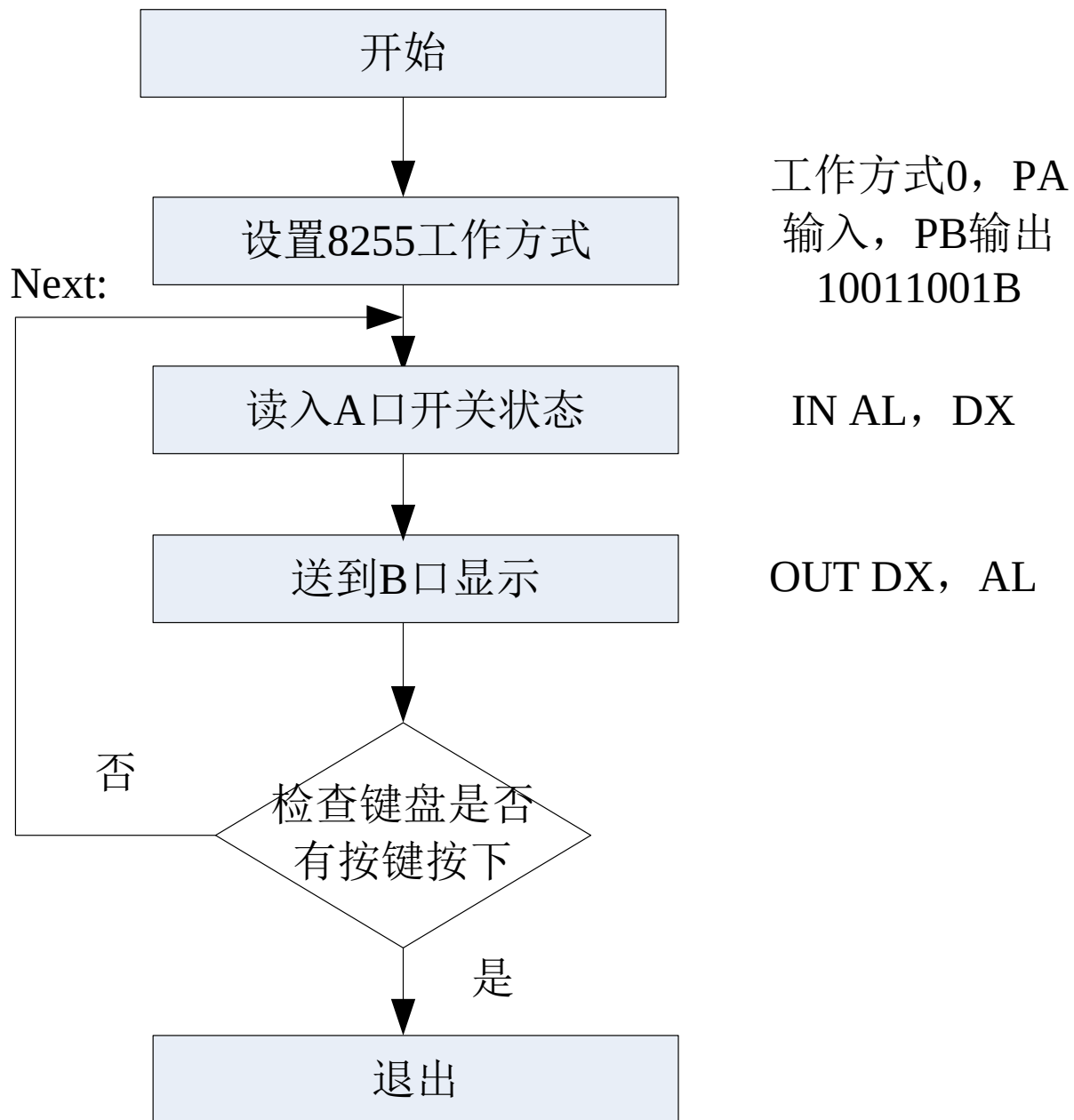
- 不等待键入
- DL=0FFH
- 键盘的键值输入AL and ZF=0
- 无键按下ZF=1 (JZ)

MOV AH, 6

MOV DL, 0FFH;

INT 21H

JZ NEXT; 无键按下跳转

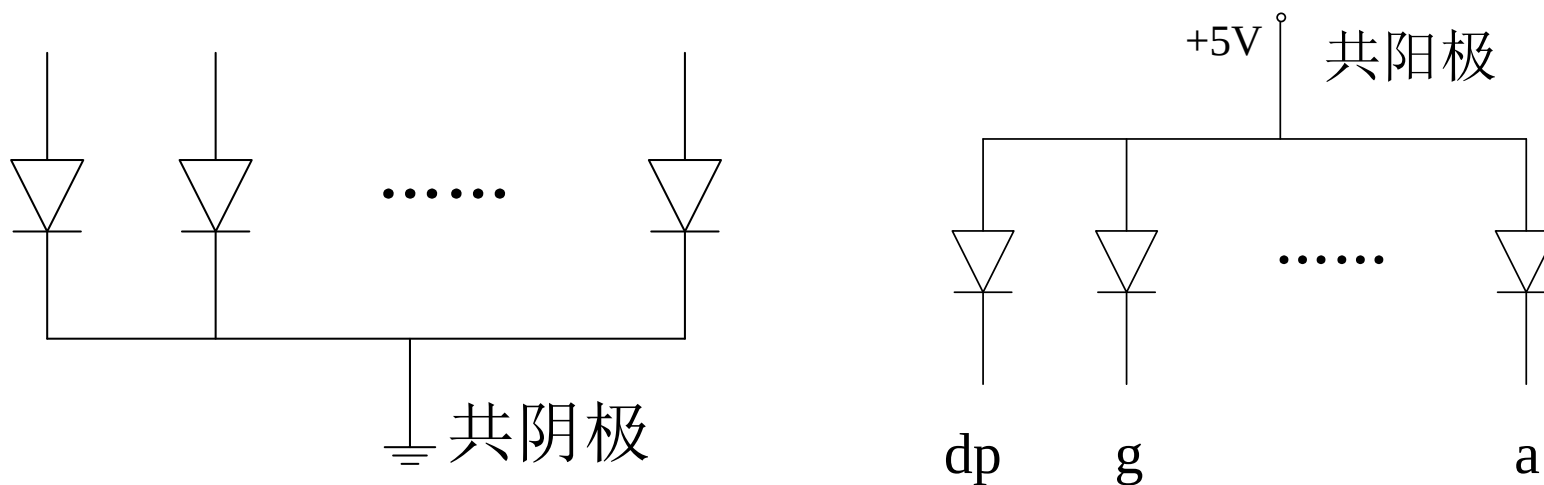
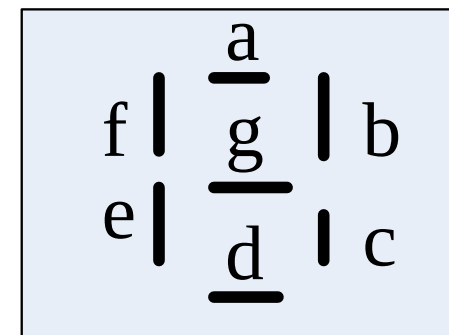


作业: 补充程序

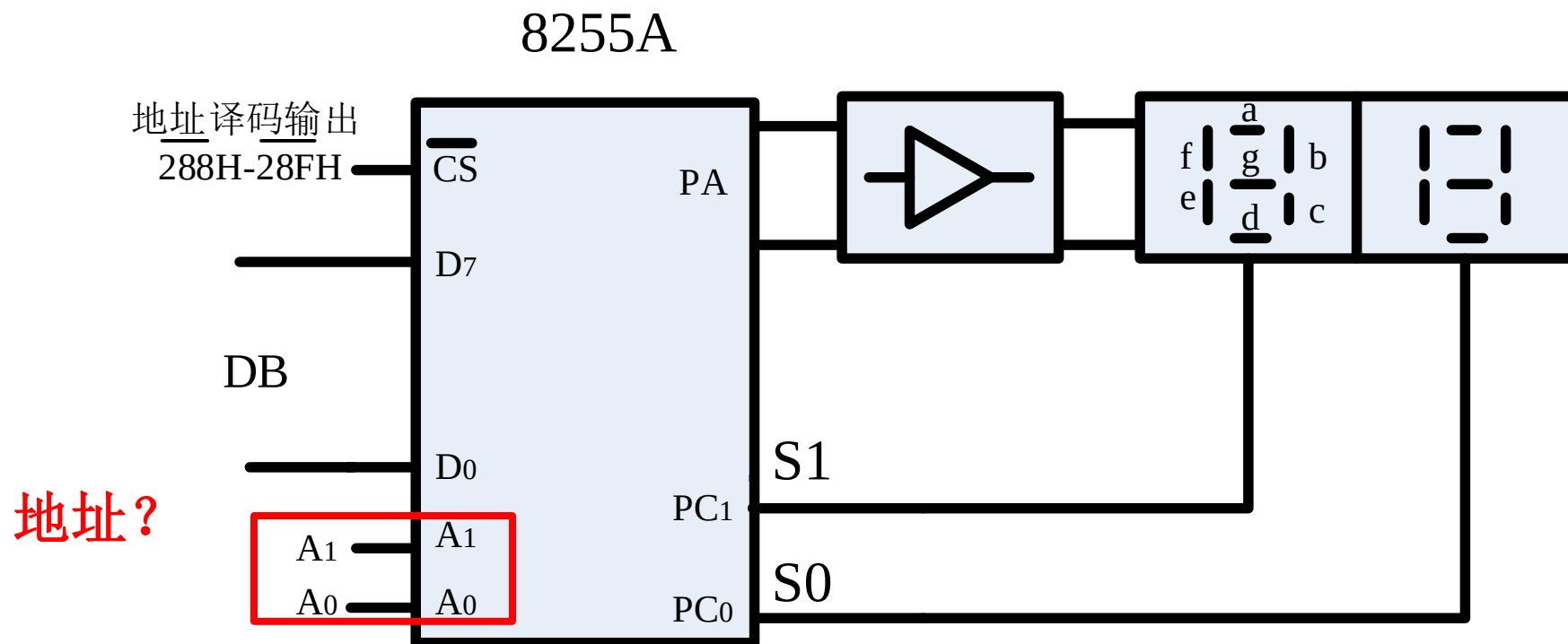
- 编程序使得**PB**口输出信号, **LED**灯按顺序点亮, 按任意键退出程序
 - 设置 AL FEH (11111110B), 使用 ROL 指令
 - 延时一段时间, 保持灯亮
 - 使用4CH功能退出

实验

- 从键盘键入2个数字并显示在数码管上
- 7段数码管
 - 共阴极Common cathode (实验采用)
 - 共阳极Common anode



硬件连线(1+7+2)



- CS 连接译码器输出，地址范围 288H~28FH
- PA6~PA0 连接七段数码管g~a (阳极)
- PC1, PC0 连接 S1, S0

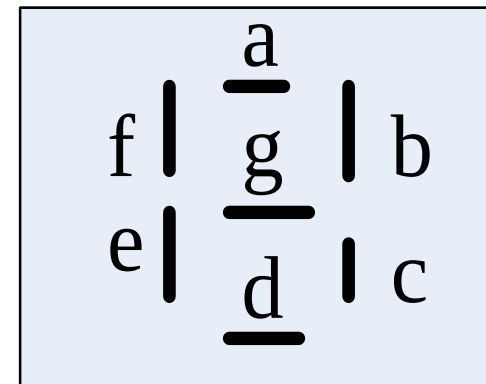
8255A地址

- 实验箱通过USB总线连接计算机, 支持即插即用
- INPORT_1 EQU 28BH
- INPORT_A EQU 288H
- INPORT_C EQU 28AH

字形码

思考：若PA6~PA0 连接a~g，
字形码为多少？

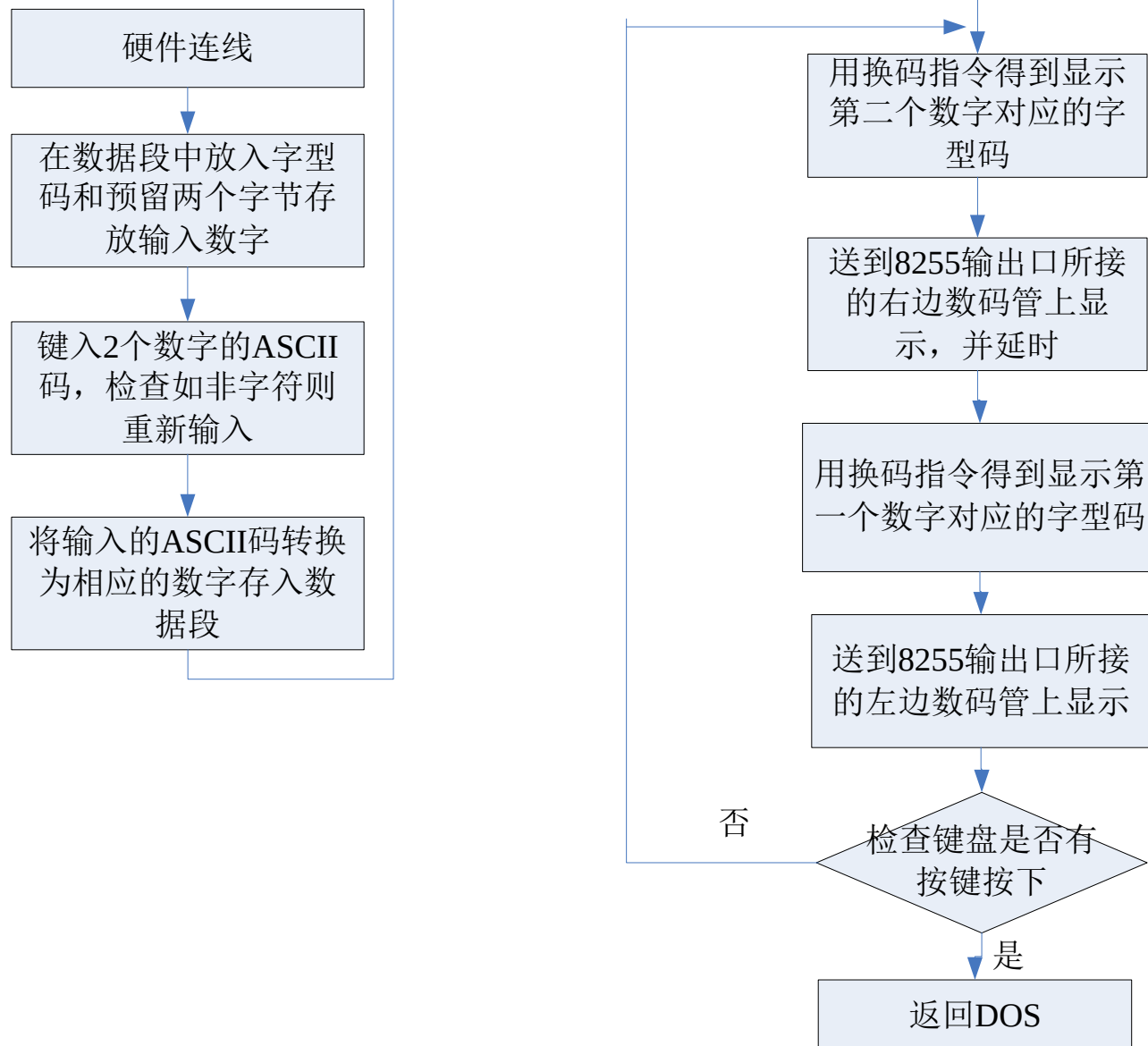
- 字形码
 - PA0~PA6 连接a~g (阳极)
 - 0, 1, ...
 - 3FH, 06H, ...



	PA7	PA6	PA5	PA4	PA3	PA2	PA1	PA0
	dp	g	f	e	d	c	b	a
3FH	0	0	1	1	1	1	1	1
06H	0	0	0	0	0	1	1	0

流程图

- 数据段定义
- 输入数据，存入数据段
 - Keyin 子程序
- 设置8255A工作方式
- 设置 A, C口输出
 - 换码指令
- 延时程序



流程图

- 数据段定义
- 输入数据，存入数据段
 - Keyin procedure
- 设置8255A工作方式

DATA SEGMENT

Y DB 3fh, 06h, 5bh, 4fh, 66h, 6dh, 7dh, 07h, 7fh, 6fh

X DB 2 DUP(?)

DATA ENDS

MOV DX, INPORT_1; 控制字地址

MOV AL, 10001010B; A口、C低四位口输出，B口输入

OUT DX, AL

流程图

- 设置 A, C口输出（若显示1、0）

NEXT:

```
MOV DX, INPORT_A; PA
MOV AL, 3FH; 显示0
```

```
OUT DX, AL
```

```
MOV DX, INPORT_C; PC
```

```
MOV AL, 10B;
```

```
OUT DX, AL
```

```
CALL DELAY; 延迟子函数
```

```
MOV DX, INPORT_A; PA
```

```
MOV AL, 06H; 显示1
```

```
OUT DX, AL
```

换码指令

```
MOV AL, X+1; 或X, 输入数字
MOV BX, OFFSET Y;
XLAT Y; AL中为对应字形码
```

```
MOV DX, INPORT_C; PC
```

```
MOV AL, 01B; 显示左边数码管
```

```
OUT DX, AL
```

```
CALL DELAY; 调用延迟子程序
```

```
MOV AH, 6
```

```
MOV DL, 0FFH;
```

```
INT 21H
```

```
JZ NEXT;
```

```
MOV AH, 4CH
```

```
INT 21H; 返回DOS
```

CPU 和外设的数据传输

数据传输方式

- IN/OUT: 8086 CPU 输入输出指令
 - 传输控制、状态和数据信息
- 3种方式: CPU与外设数据传输
 - **编程控制方式**: 执行指令来完成
 - 直接传输方式
 - 查询方式
 - **中断方式**
 - **DMA 方式** (Direct Memory Access)

软件控制

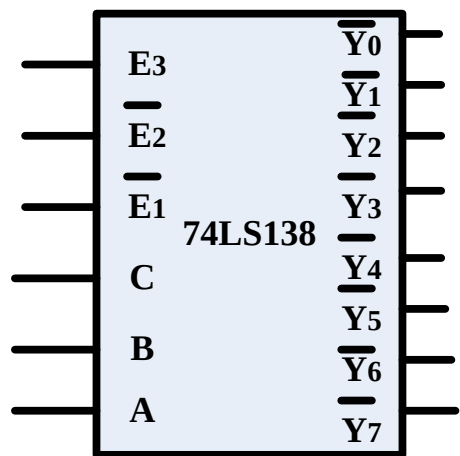
硬件控制

直接传输方式

- 直接传输方式
 - 外设始终准备好数据传输，不需要检查外设当前状态
 - 直接执行IN/OUT 指令
- 典型电路图

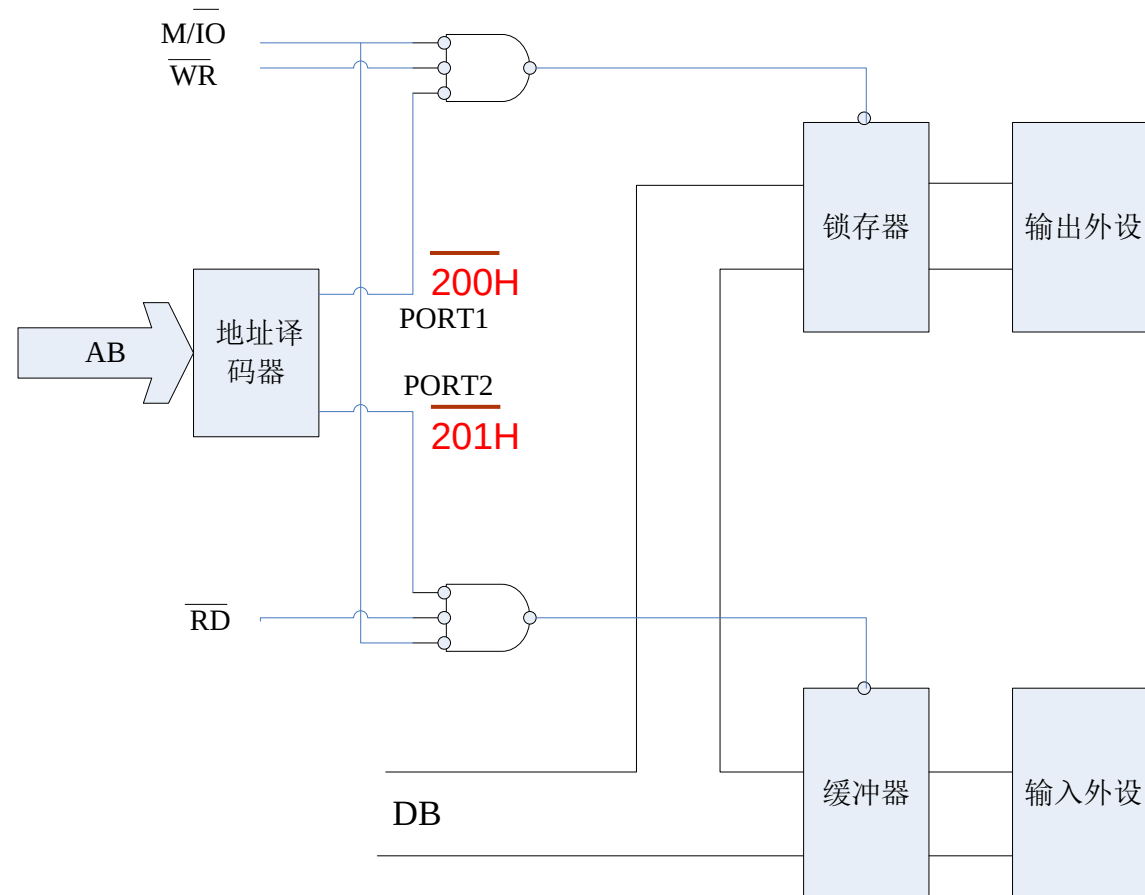
直接传输方式典型电路

- 输入缓冲
- 输出锁存
- 地址译码器
- 编程
 - Eg. 与 [SI] 交换数据



```
MOV DX, 200H
MOV AL, [SI]
OUT DX, AL
```

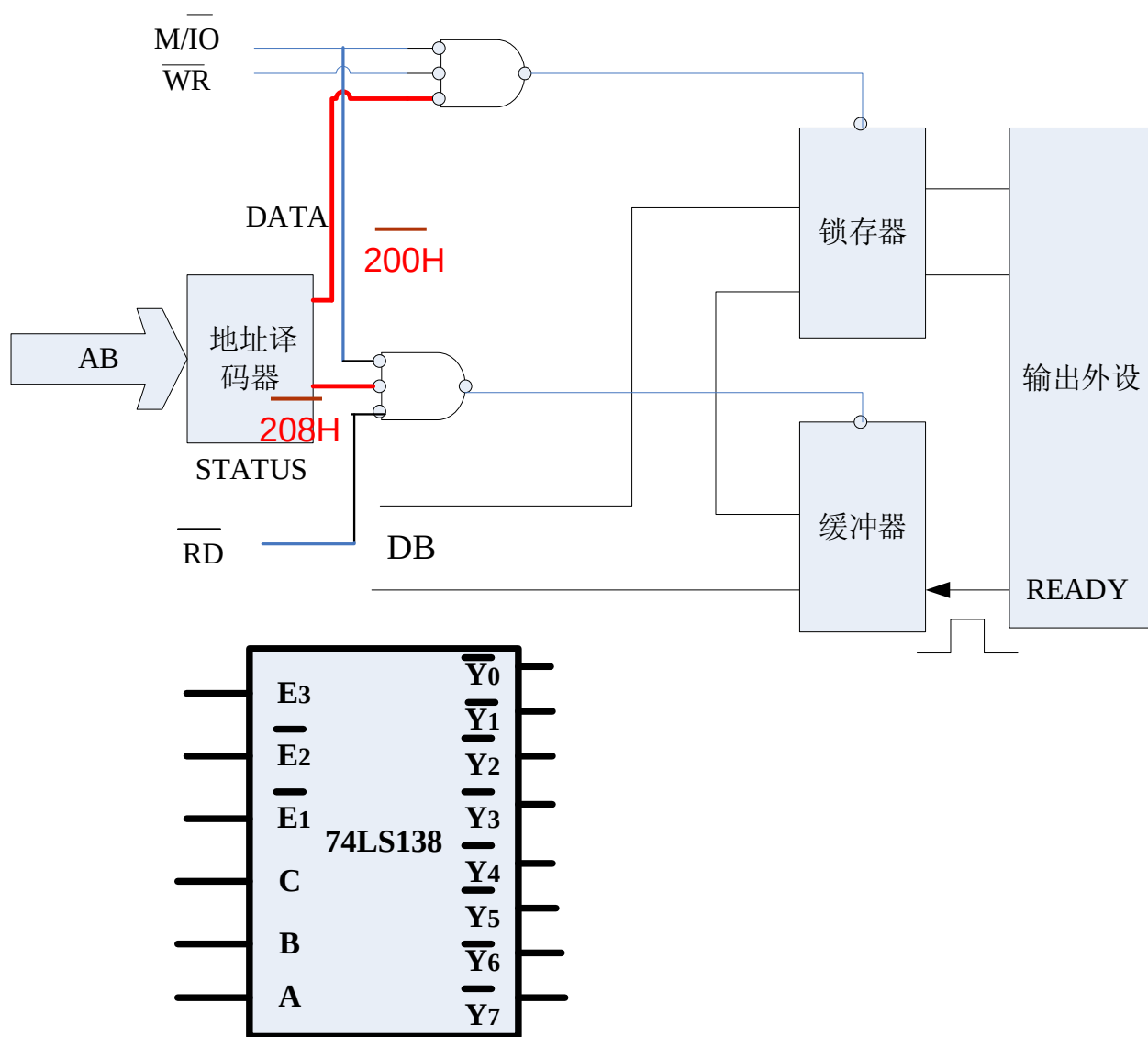
```
MOV DX, 201H
IN AL, DX
MOV [SI], AL
```



查询方式

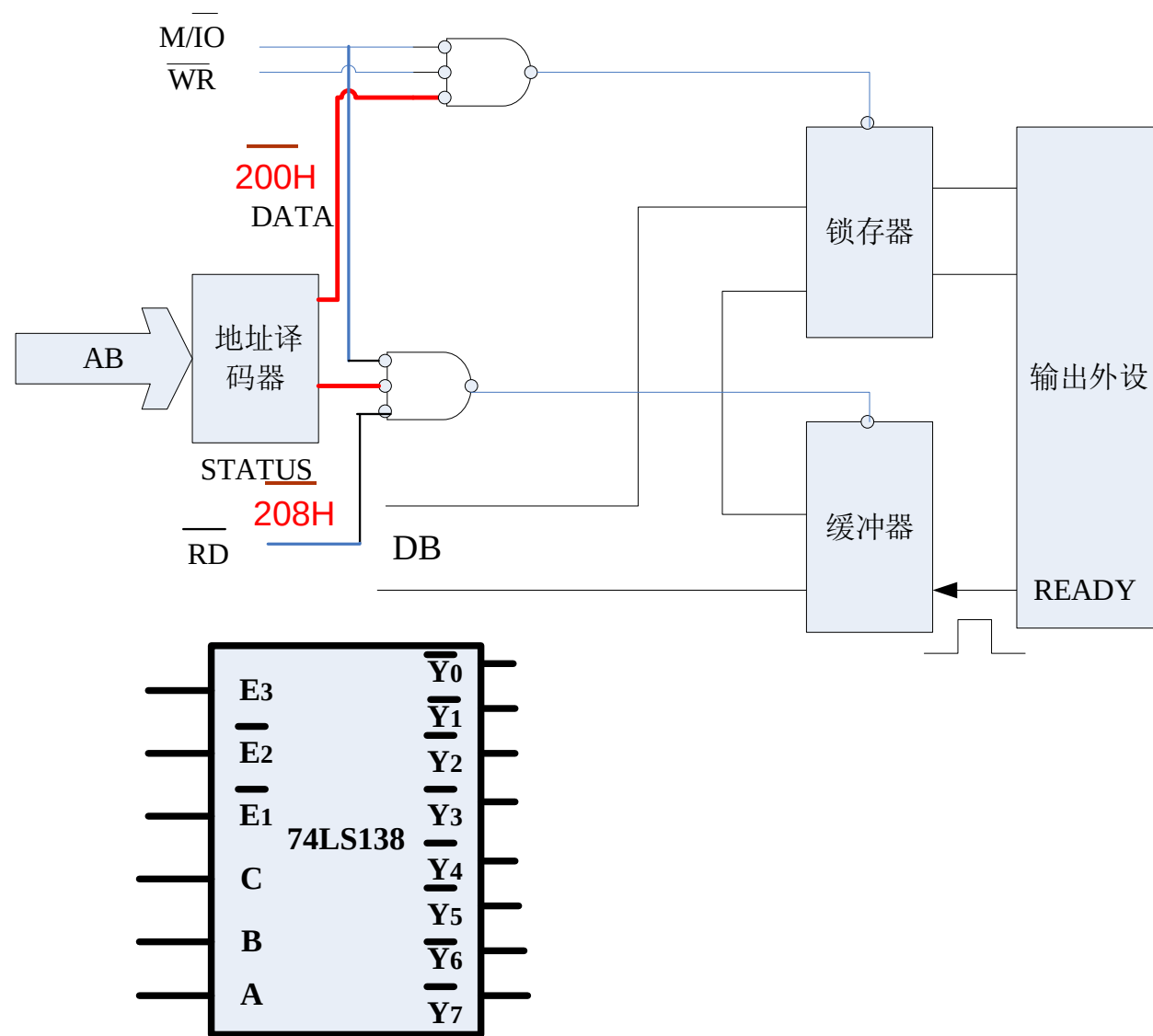
- 传输前通过状态端口查询外设状态
- 若外设准备好则启动传输，否则CPU一直查询直至外设准备好
 - CPU在外设准备好之前一直查询
 - Eg: 打印机，低速工作，从状态口提供状态信息

查询方式输出电路



- 外设状态线: READY—**D₀**
- 输出接口电路（2个地址）
 - 状态端口(IN)
 - 数据端口(OUT)
- 数据口和状态口有不同地址
- **编程**
 - Eg. 输出[BX]中数据
 - **D₀**: 表示状态

查询方式输出电路



MOV DX, 208H;

WAIT1:

IN AL, DX

TEST AL, 1

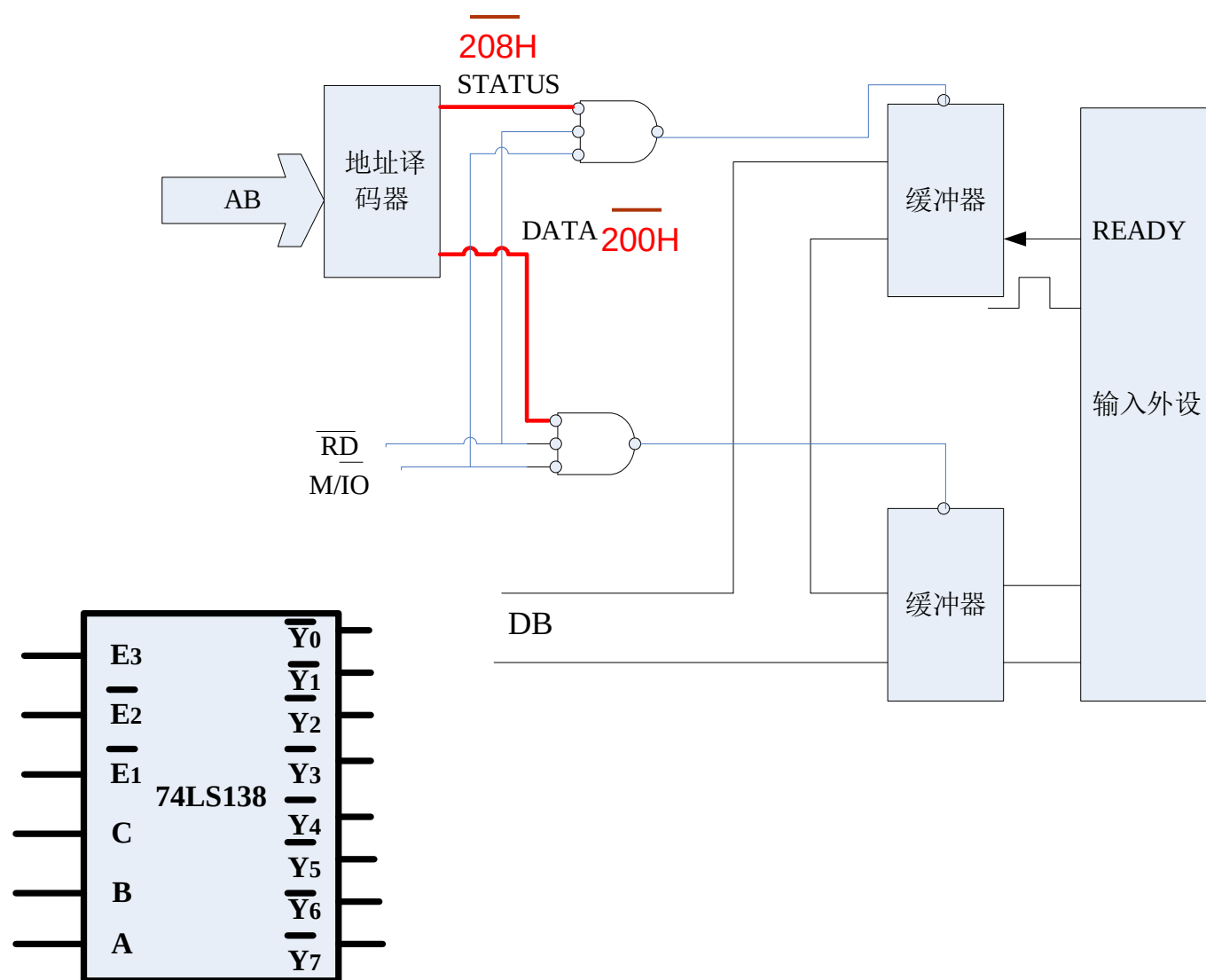
JZ WAIT1;

MOV DX, 200H;

MOV AL, [BX]

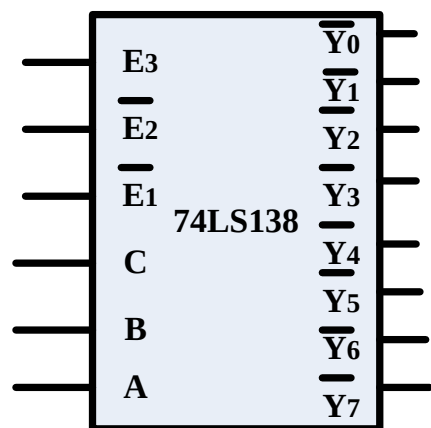
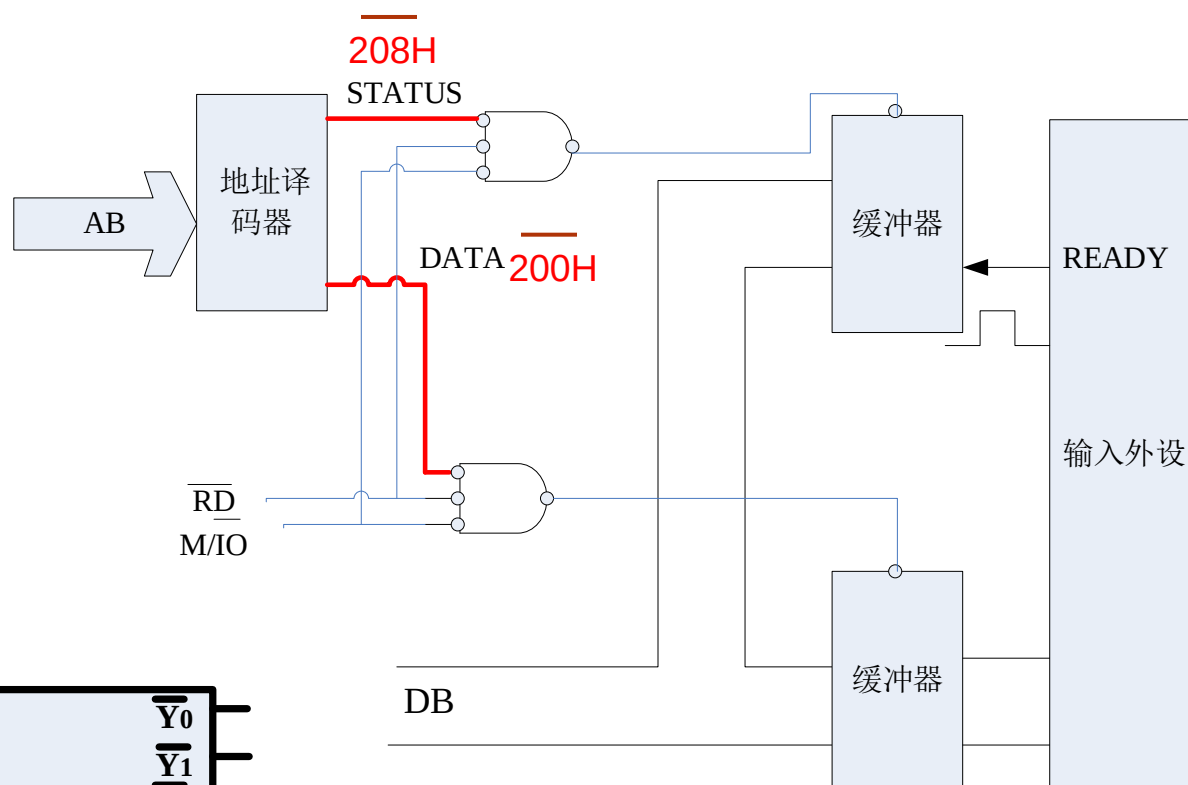
OUT DX, AL

查询方式输入电路



- 外设状态线:
READY—D₀
- 输入接口电路
 - 状态端口(IN)
 - 数据端口(IN)
- 数据口和状态口有不同地址
- **编程**
 - **数据读入[BX]**

查询方式输入电路



MOV DX, 208H

WAIT1:

IN AL, DX

TEST AL, 1

JZ WAIT1;

MOV DX, 200H;

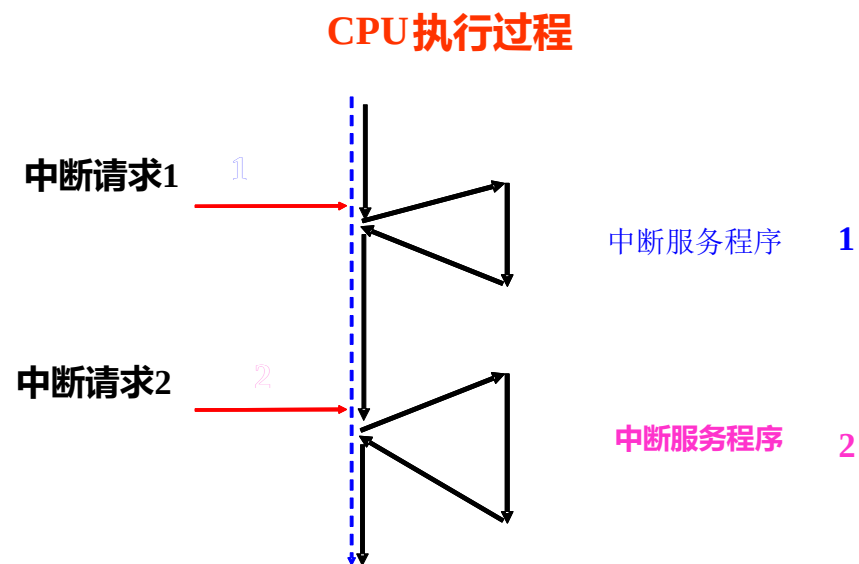
IN AL, DX

MOV [BX], AL

• CPU 效率低（当外设未准备好）

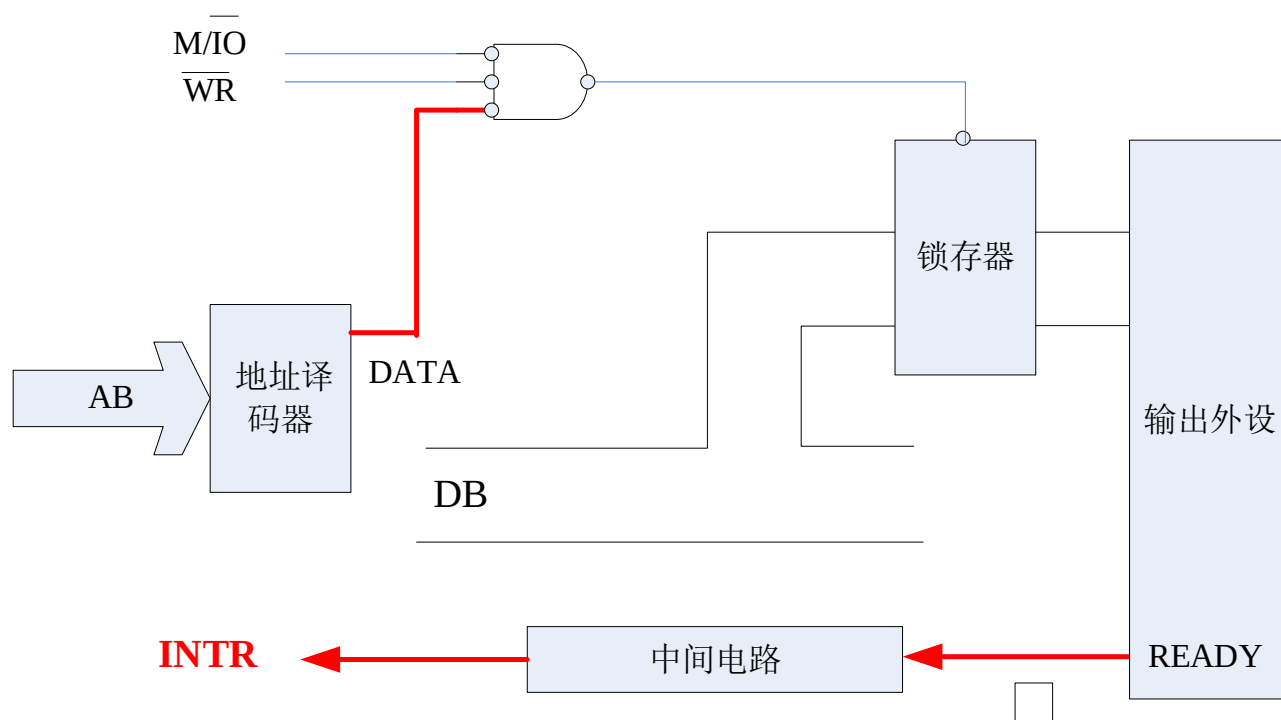
中断方式

- 发送中断请求
 - CPU 执行主程序; 当输入和输出设备准备好, 发送中断请求给CPU
- CPU响应请求
 - CPU 挂起当前程序, 转而执行中断服务程序, 处理外设请求
- 输入输出结束
 - CPU返回被打断处继续执行
- 中断输出电路



中断输出电路

- 中断服务程序(output)
- [BX]



中断服务子程序:

INTTEST PROC

.....

MOV DX, DATA

MOV AL, [BX]

OUT DX, AL

.....

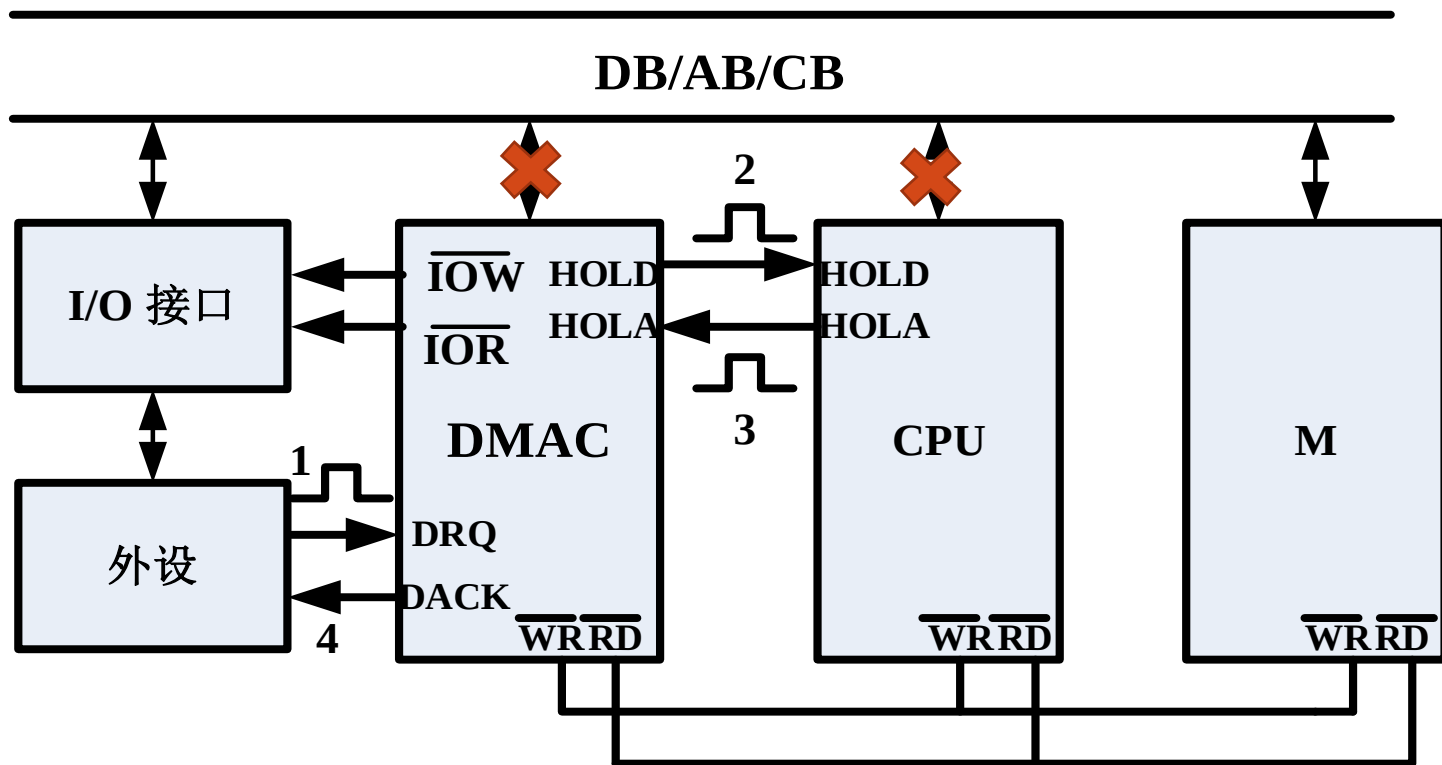
IRET; 中断返回

INTTEST ENDP

DMA

- 内存和外设之间有大量数据传输
 - 中断方式: 入栈, 出栈, 清空指令队列 etc.
 - 较为低效
- DMA controller (DMAC)
 - DMA控制芯片
 - Intel 8237A
 - 硬件控制的数据传输
- 连接电路

连接电路



- HOLD
 - 总线保持请求
- HOLA
 - 总线保持相应

- DMAC 和 CPU: 共享对内存的访问
- DMAC 接管总线: DB/AB/CB
- 理解书中总线控制的三个开关

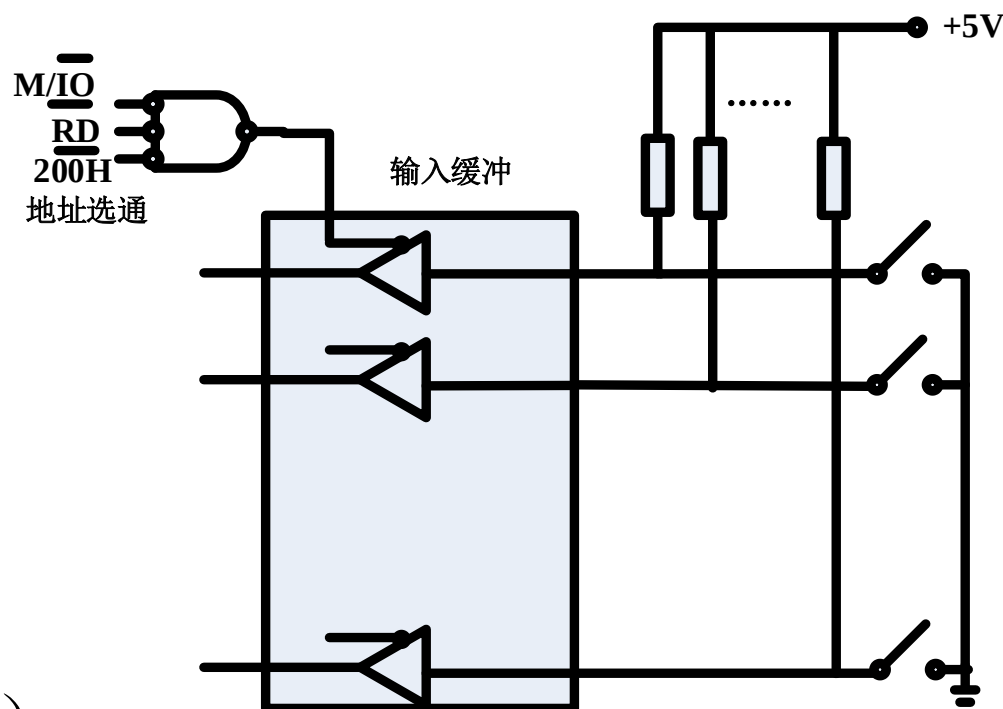
8255A应用: 键盘接口

- 键盘是最常用的外设，由多个开关组合而成，压下一个键盘等于压下一个开关

MOV DX, 200H

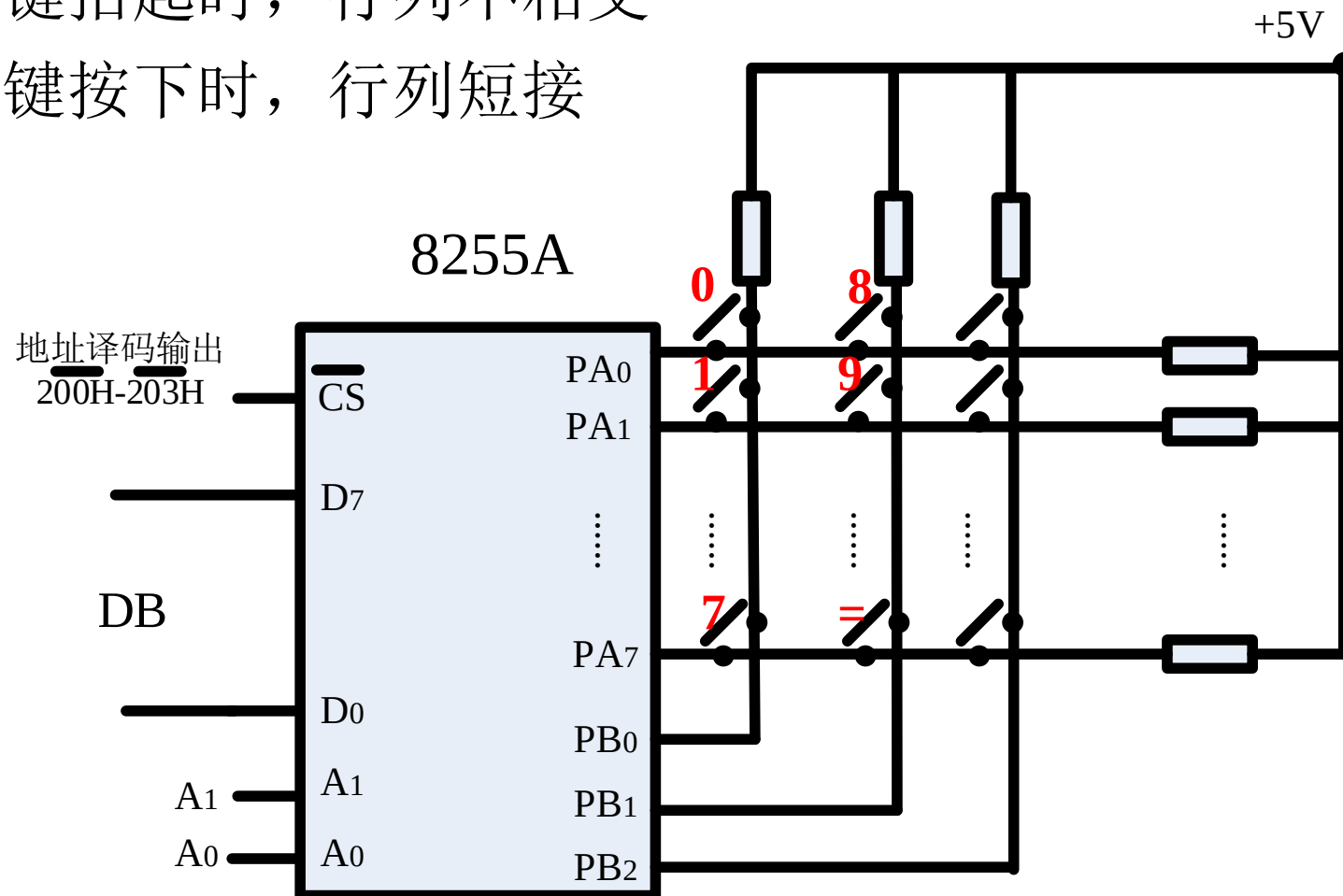
IN AL, DX

- 1个按键对应
 - 1个口线（输入端口和输入线）
 - 过于复杂浪费



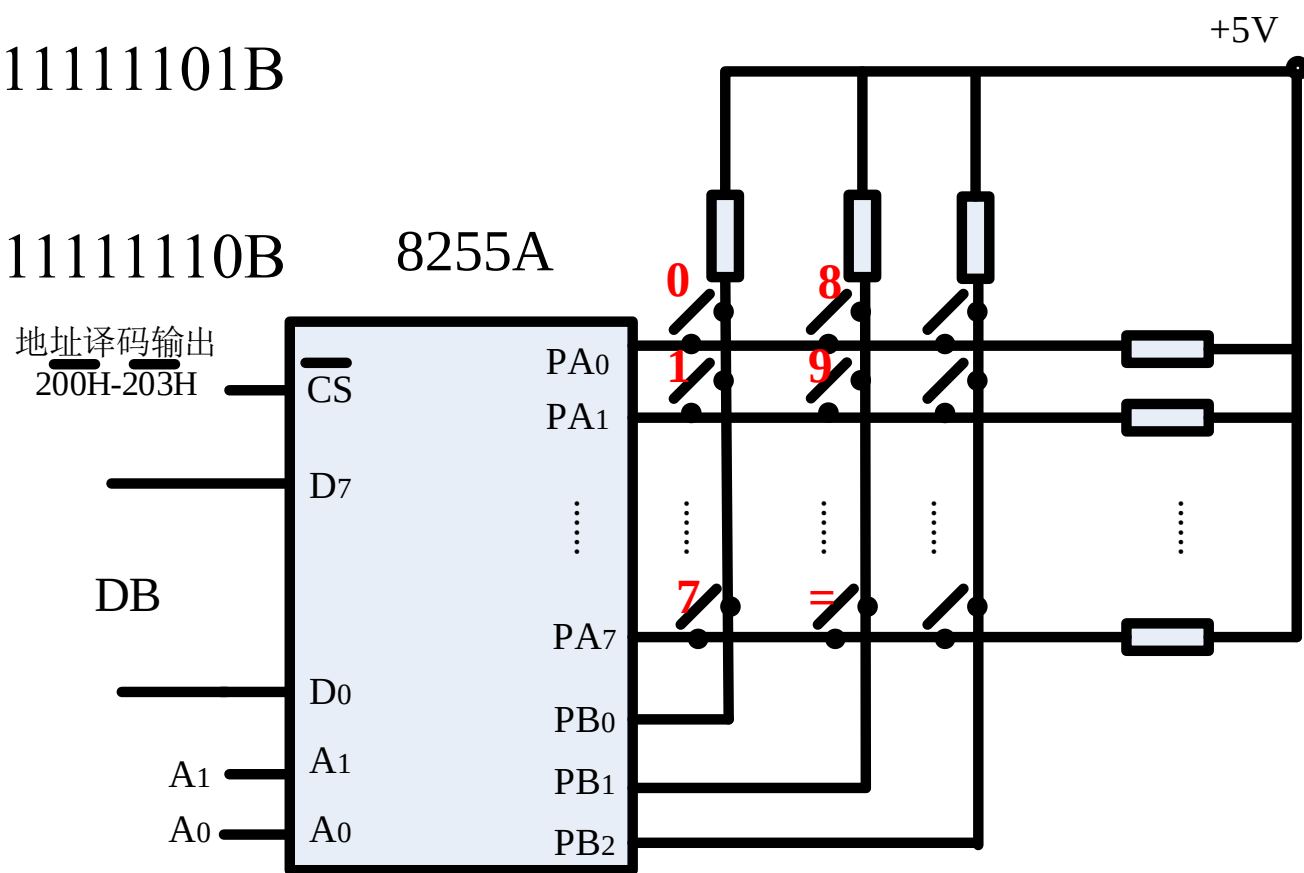
键盘阵列

- 当按键抬起时，行列不相交
- 当按键按下时，行列短接



键盘阵列

- 行列反转法Column and row switch
 - PA输出全0，PB读入作为列码
 - '0': 11111110B '1': 11111110B '8': 11111101B
 - PB输出全0，PA读入作为行码
 - '0': 11111110B '1': 11111101B '8': 11111110B
 - 键值: 列码+行码
 - 对应不同按键



键值表

- 第一列

- 第二列

按键	键值(二进制)	键值(H)
‘0’	1111111 0 1111111 0 B	FEFEH
‘1’	1111111 0 111111 0 1B	FEFDH
‘2’	1111111 0 11111 0 11B	FEFBH
‘3’	1111111 0 1111 0 111B	FEF7H
‘4’	1111111 0 111 0 1111B	FEEFH
‘5’	1111111 0 11 0 11111B	FEDFH
‘6’	1111111 0 1 0 111111B	FEBFH
‘7’	1111111 0 0 1111111B	FE7FH
按键	键值(二进制)	键值(H)
‘8’	111111 0 1 1111111 0 B	FDFEH
‘9’	111111 0 1 111111 0 1B	FDFDH
...	111111 0 1 11111 0 11B	FDFBH
....	111111 0 1 1111 0 111B	FDF7H
....	111111 0 1 111 0 1111B	FDEFH
....	111111 0 1 11 0 11111B	FDDFH
....	111111 0 1 1 0 111111B	FDBFH
....	111111 0 1 0 1111111B	FD7FH

键盘阵列

DATA SEGMENT

Value DW 0FEFEH, 0FEFDH, 0FEFBH, 0FEF7H,
0FEEFH, 0FEDFH, 0FEBFH, 0FE7FH,
0FD FEH, 0FD FDH, 0FD FBH, 0FEF7H,
.....

Char DB '0', '1', '2', '3',

DATA ENDS

键盘阵列

控制口地址：203H

A口地址：200H

B口地址：201H

- 8255A 初始化 PA: 输出 PB: 输入 10001011B
- PA输出全0，PB读入作为列码
 - 列码为0FFH无键按下，不为0FFH则延时防抖动后重新读入列码
- 8255A 重新初始化 PB:输出 PA:输出 10011001B
- PB输出全0，PA读入作为行码
- 键值:列码+行码
- 比较查表
- 显示键值

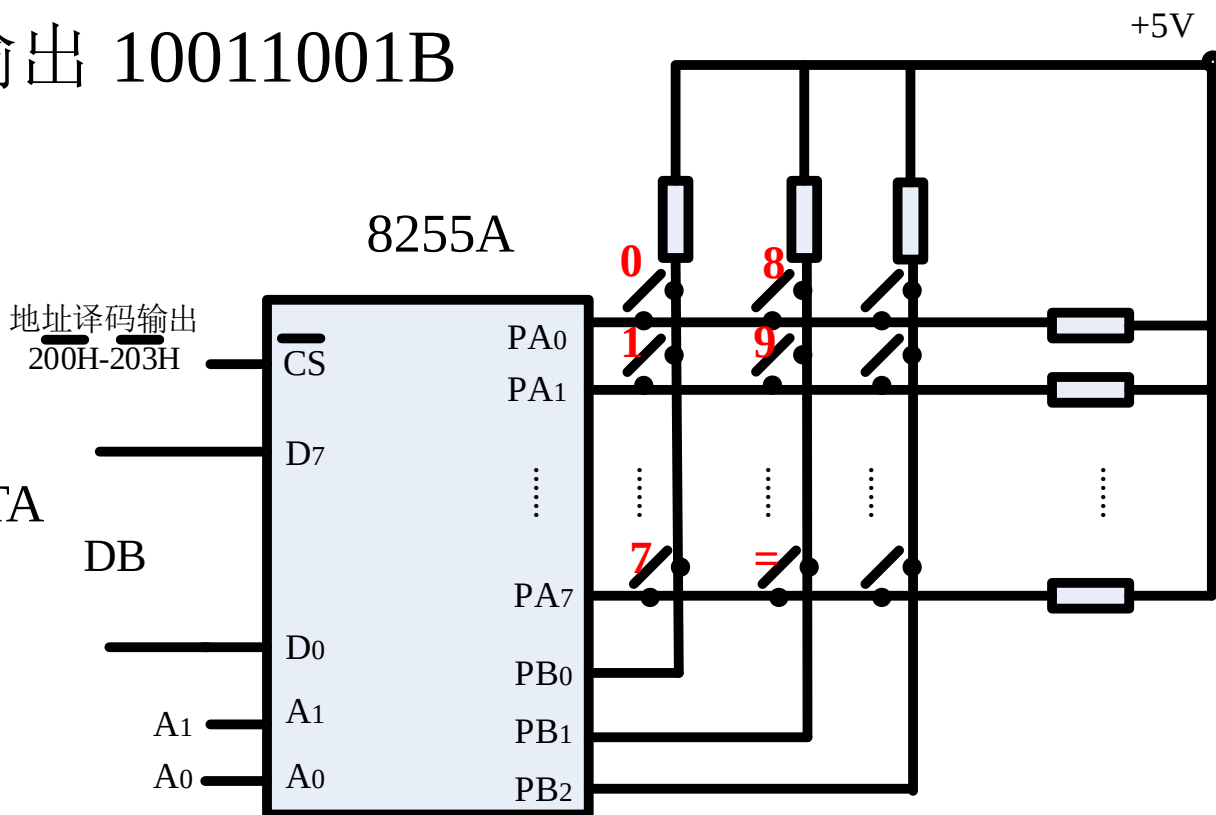
CODE SEGMENT

ASSUME CS:CODE, DS:DATA

MAIN PROC

MOV AX, DATA;使用数据段

MOV DS, AX



键值识别代码

AX中为列码AH+行码AL，可查表得到对应按键

NEXT0:

```
MOV DX, 203H; 8255A控制字
MOV AL, 10001011B; A输出B输入
OUT DX, AL
```

NEXT1:

```
MOV AL, 0
MOV DX, 200H
OUT DX, AL; PA口全输出0
MOV DX, 201H
IN AL, DX; 读入B口数据
CMP AL, 0FFH
JE NEXT1;
MOV CX, 1000H
```

NEXT2: LOOP NEXT2; 延时防抖动

```
IN AL,DX;信号稳定后再次读入
MOV AH, AL; 保存B口列码
```

```
MOV DX, 203H; 行列反转
MOV AL, 10011001B; A输入B输出
OUT DX, AL
MOV AL, 0
MOV DX, 201H
OUT DX, AL; PB输出0
MOV DX, 200H; A口地址
IN AL, DX; A口读入行码
```

键值识别代码

```
MOV SI, OFFSET Value
MOV DI, OFFSET Char
MOV CX, 24; 准备循环比较键值
```

NEXT3:

```
CMP AX, [SI]; DS:SI
JE NEXT4; 相等则转去显示
ADD SI, 2; 指向Value下一个字
INC DI; 指向Char下一个字节
LOOP NEXT3
JMP NEXT0; 没有匹配, 重新开始
```

查表得到对应按键

NEXT4:

```
MOV DL, [DI]
MOV AH, 2
INT 21H; 显示当前键值
MOV DL, 0FFH; 检查标准键盘
MOV AH, 6
INT 21H;
JZ NEXT0 ; 无按键按下
```

EXIT:

```
MOV AH, 4CH
INT 21H
```

.....

扫码答题



Review of chapter 6

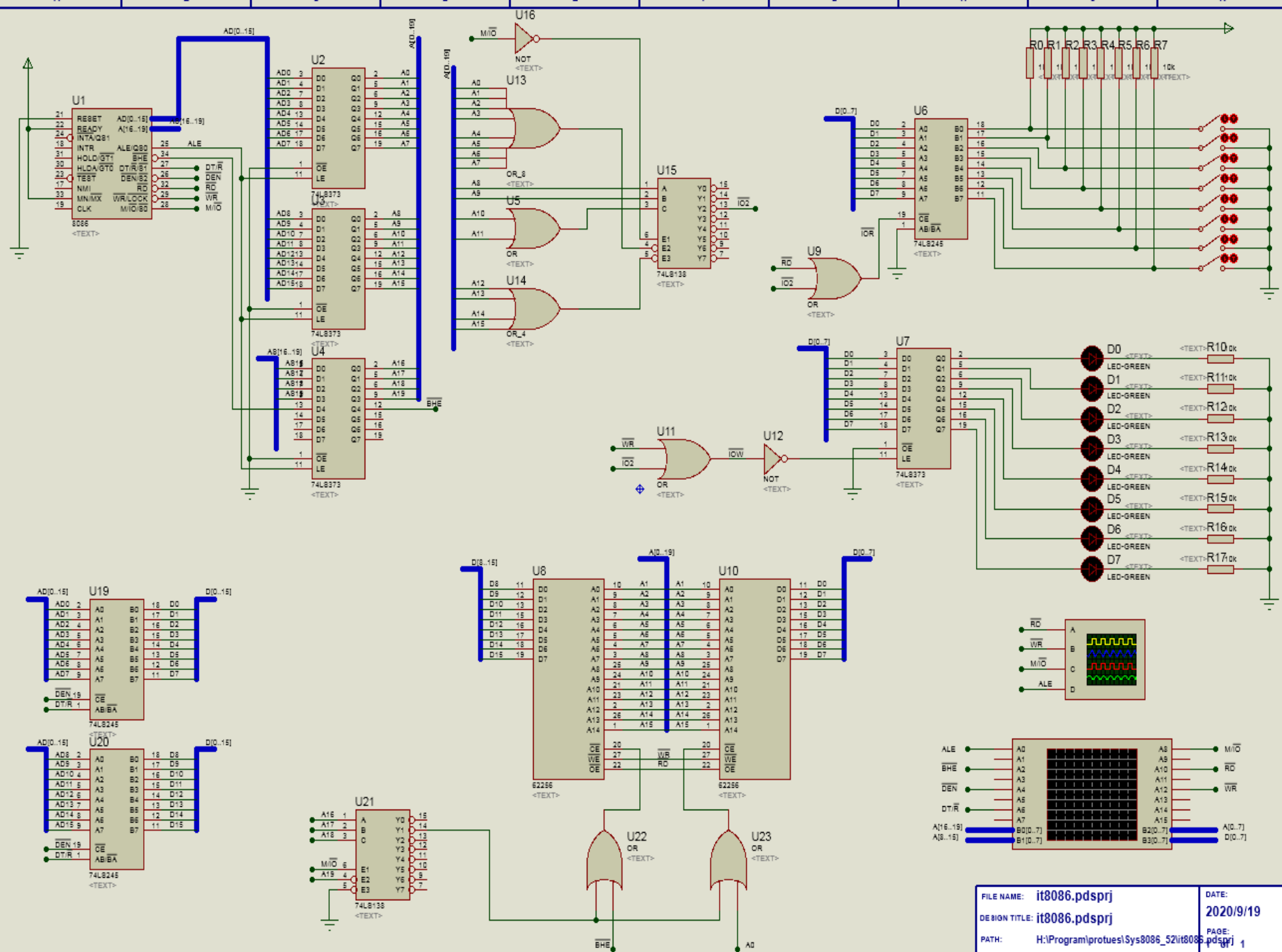
- I/O 接口
 - 概念：数据输入输出，接口，端口，接口作用，IO信号类型，数据口，状态口，命令口
 - 基本数据传输方式：程序控制(数据传输, 查询), 中断, DMA
 - IN/OUT 指令
- 8255A
 - 8255A方式1
 - 初始化及输入输出编程
 - 8255A：键盘控制与显示

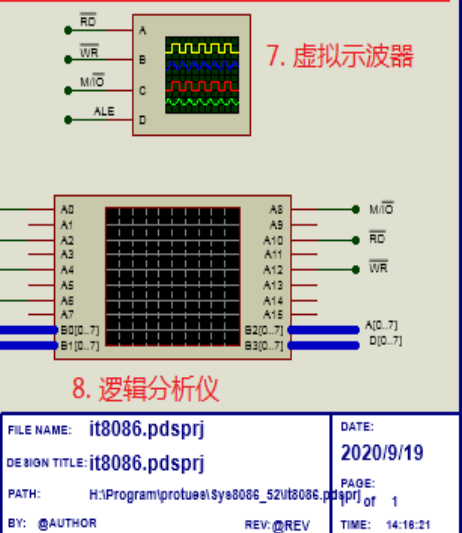
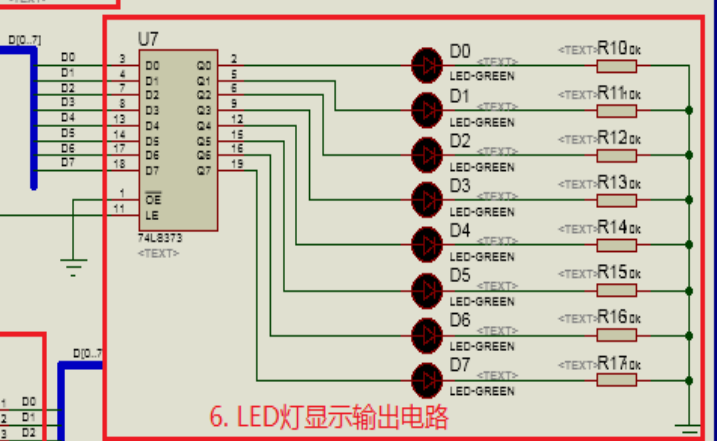
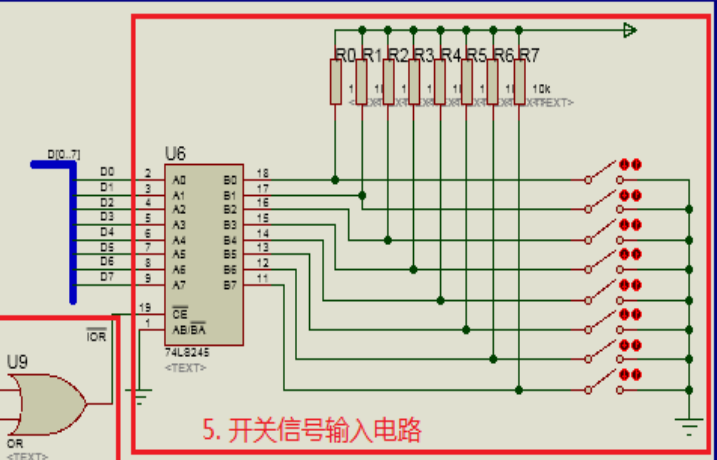
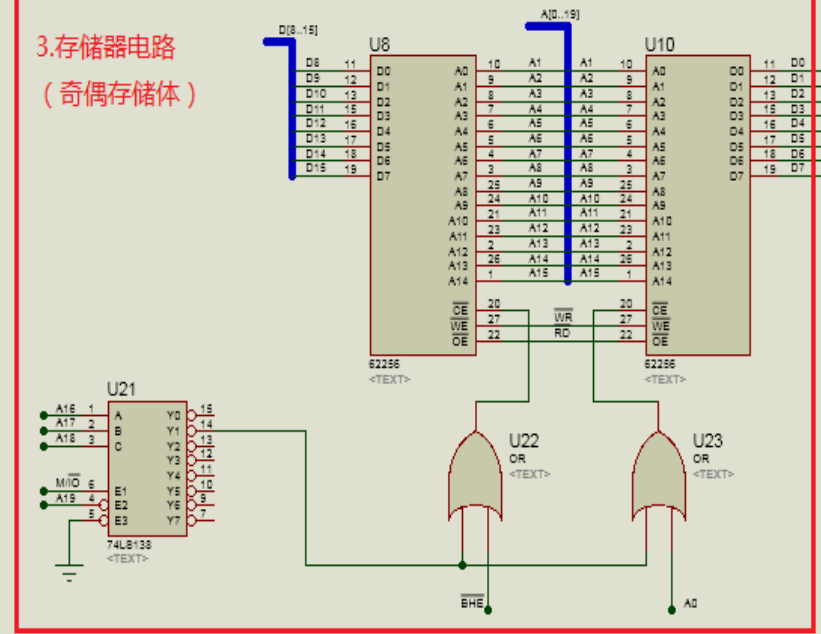
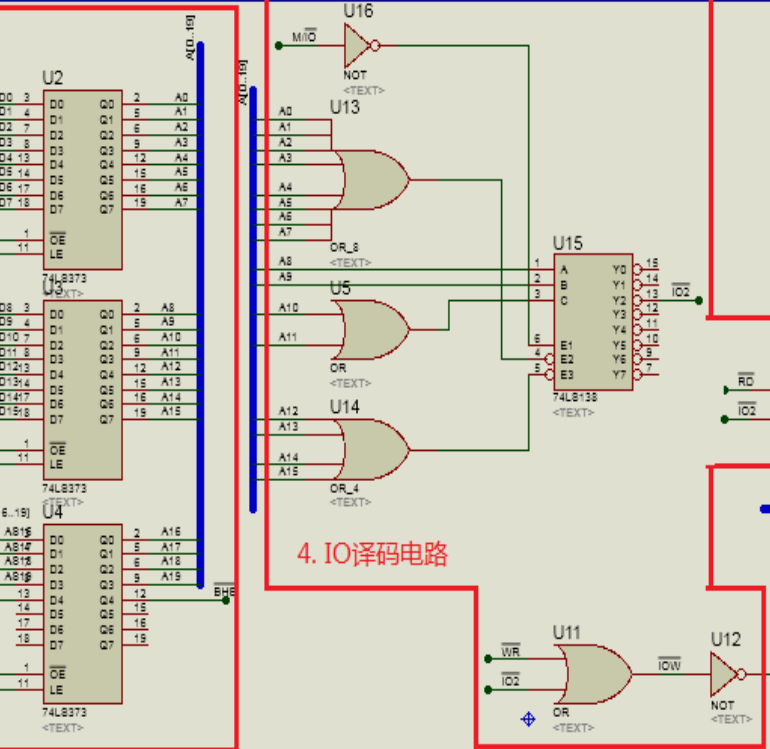
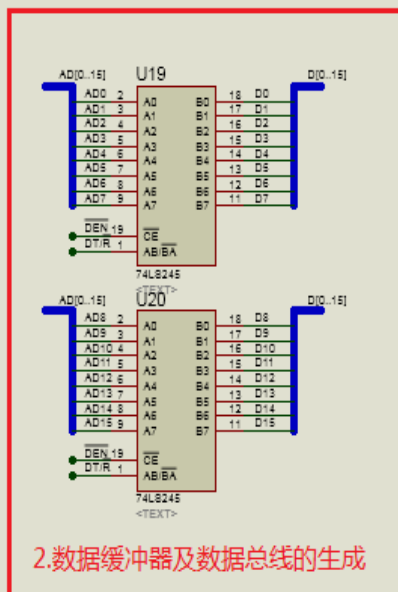
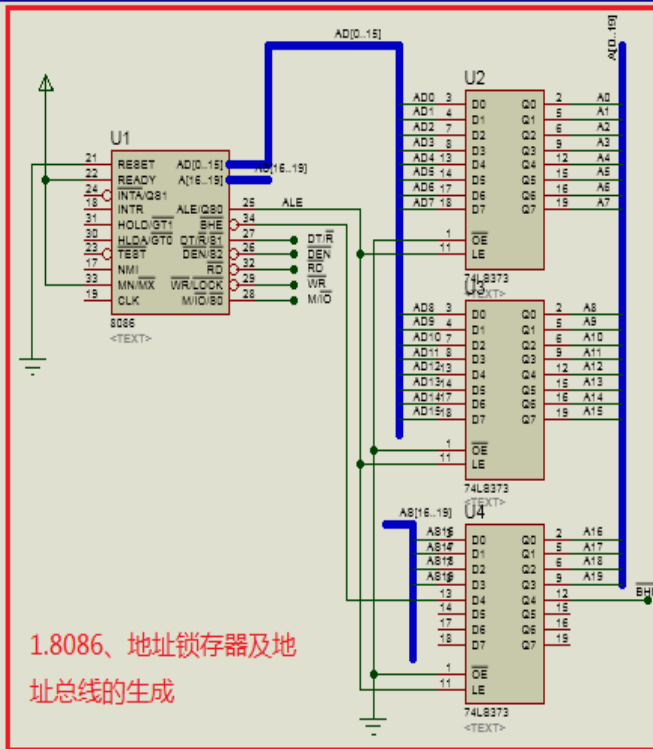
EDA电路原理图及仿真Project

搭建8086最小模式系统

项目要求

- 在Proteus软件中搭建电路原理图
 - 附有所需的元器件列表，可以在Proteus中查找并添加到自己的元器件列表中
- 将提供的汇编程序复制到Proteus项目中的源程序中（默认是main.asm），编译、仿真运行
- 查找资料，添加虚拟示波器或者逻辑分析仪，观察和记录指令执行时的地址总线、数据总线、ALE、 $\overline{\text{BHE}}$ 、 $\overline{\text{RD}}$ 、 $\overline{\text{WR}}$ 、 $\overline{\text{M/IO}}$ 等信号的波形
- 作业提交：对电路搭建过程中遇到的问题、解决方法、仿真运行进行总结和分析。1个月内将分析总结报告、Proteus项目打包压缩后提交到spoc



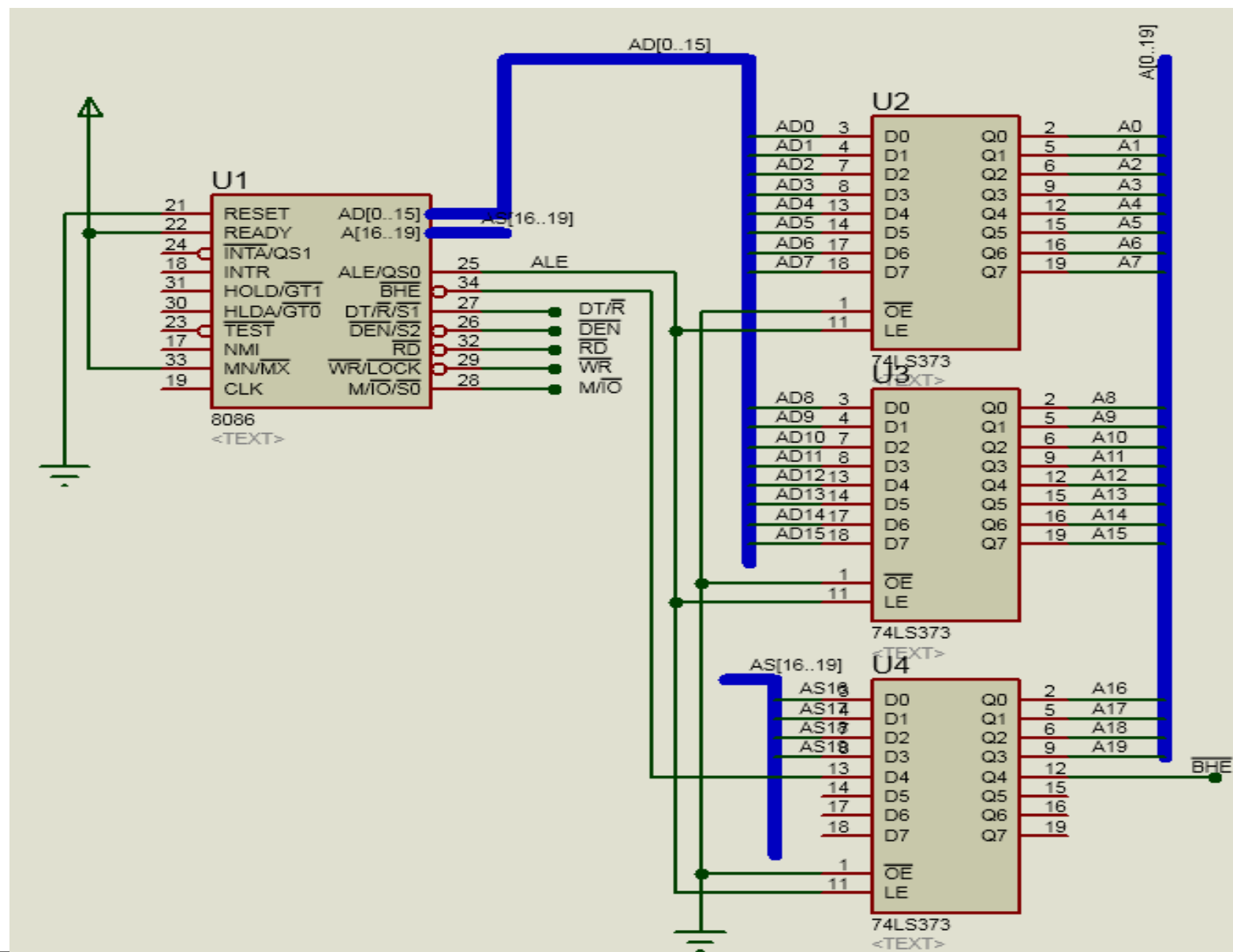


8. 逻辑分析仪

FILE NAME: it8086.pdsprj	DATE: 2020/9/19
DESIGN TITLE: it8086.pdsprj	PAGE: 1
PATH: H:\Program\proteus\Sys8086_52\it8086.pdsprj	REV: @REV
BY: @AUTHOR	TIME: 14:18:21

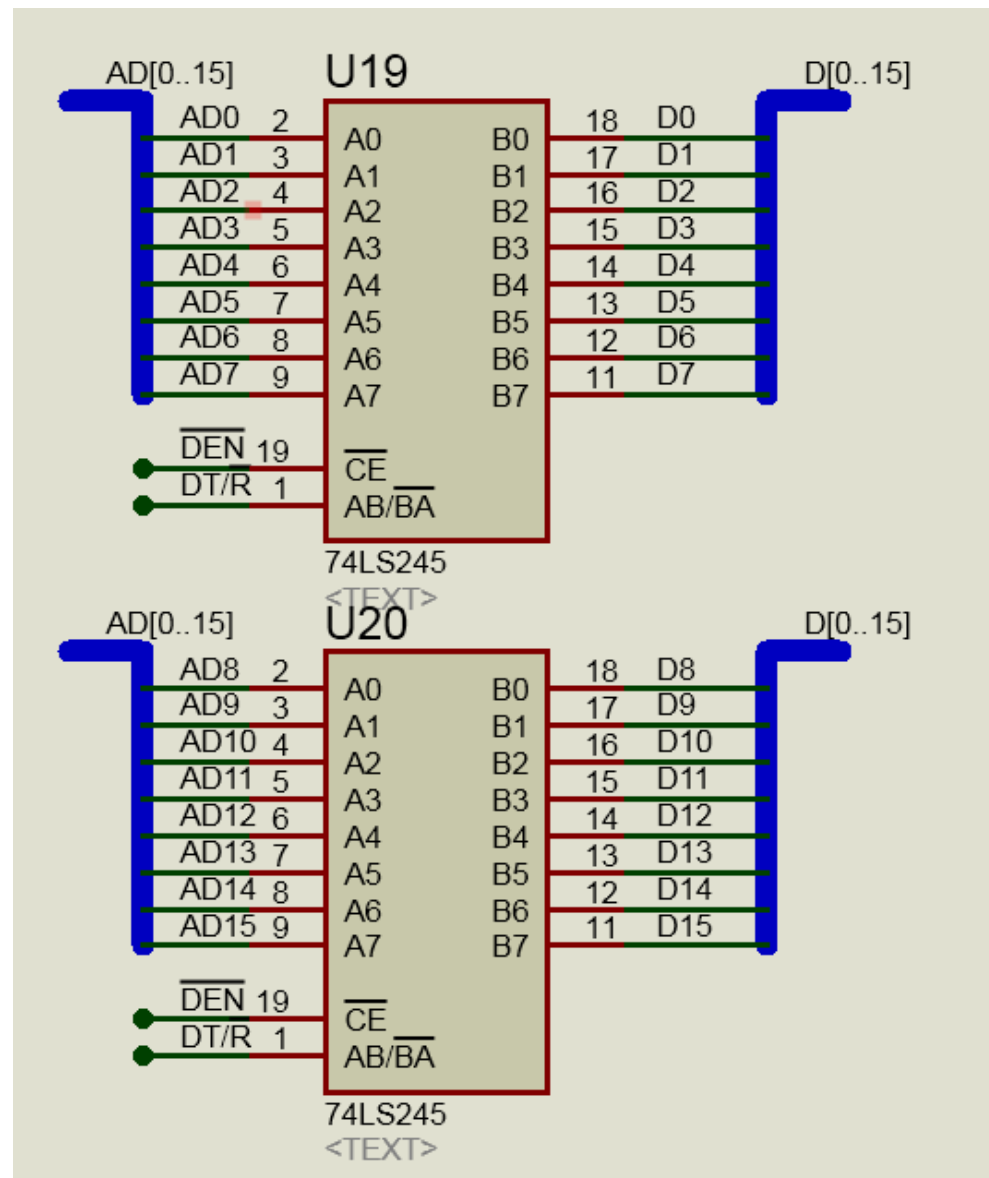
1. 8086地址锁存器及地址总线的生成

- 3个74LS373锁存器锁存21个信号
- 形成20位地址总线

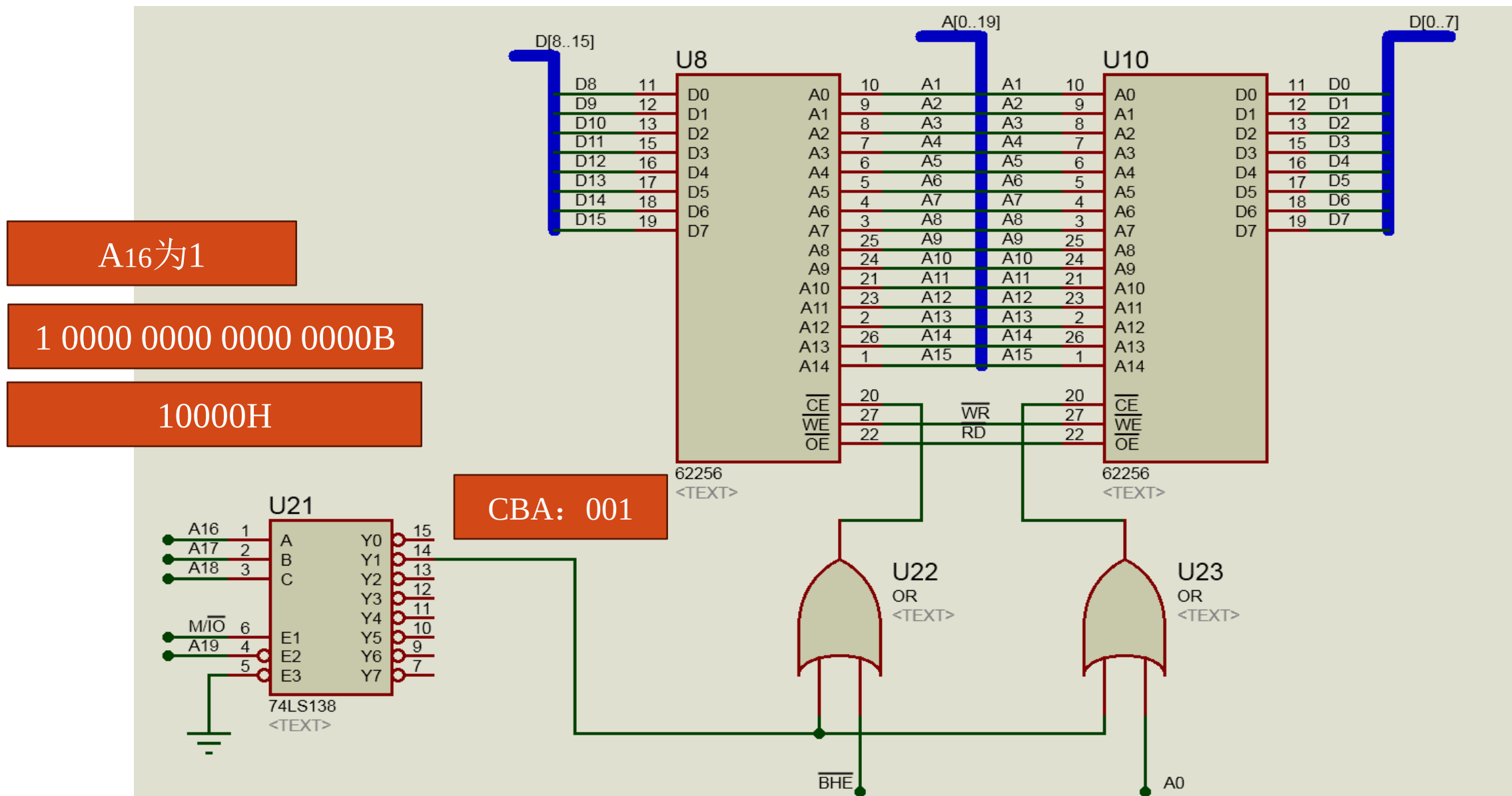


2.数据缓冲器及数据总线的生成

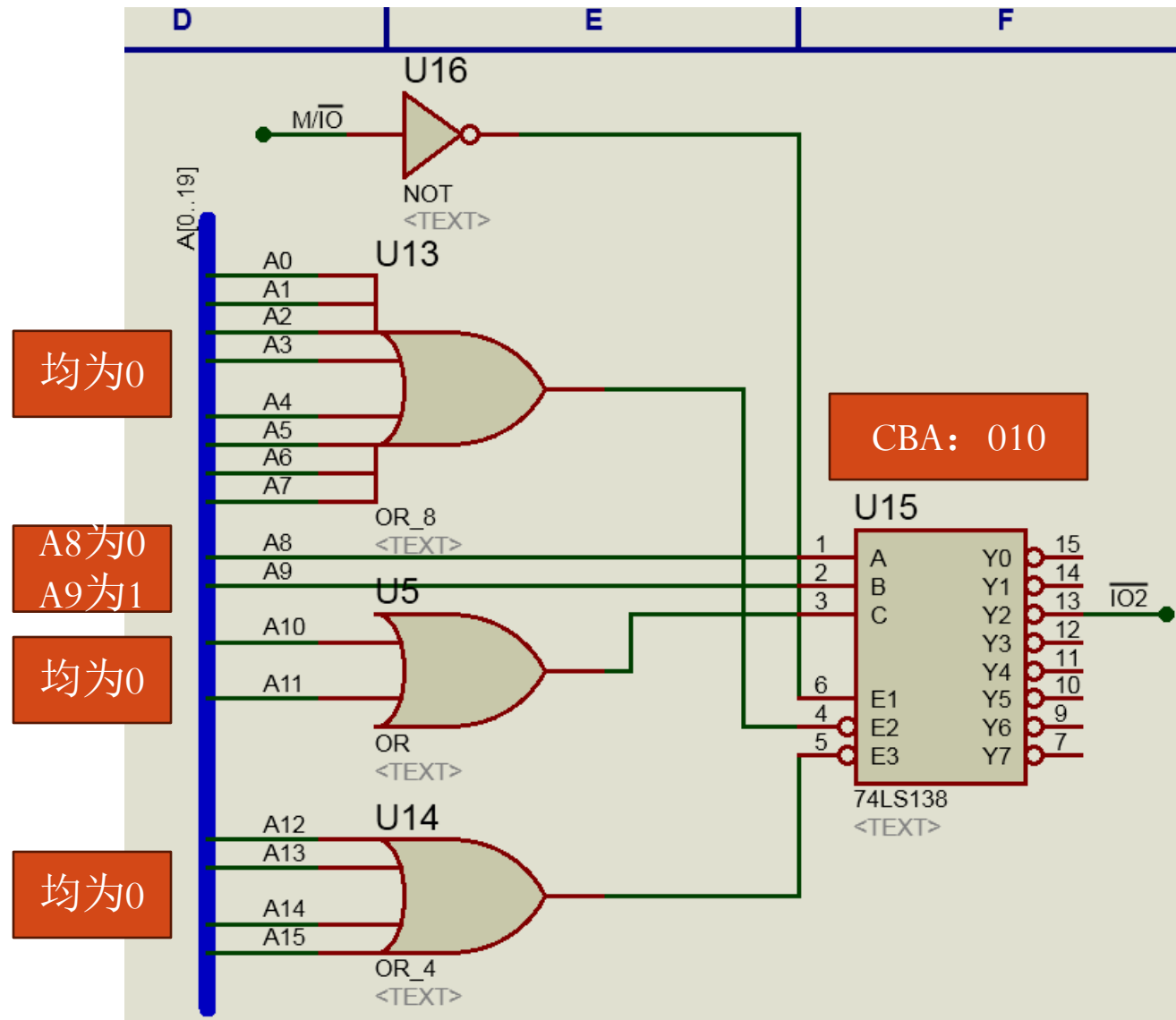
- 2个74LS3245双向缓冲形成16位数据总线



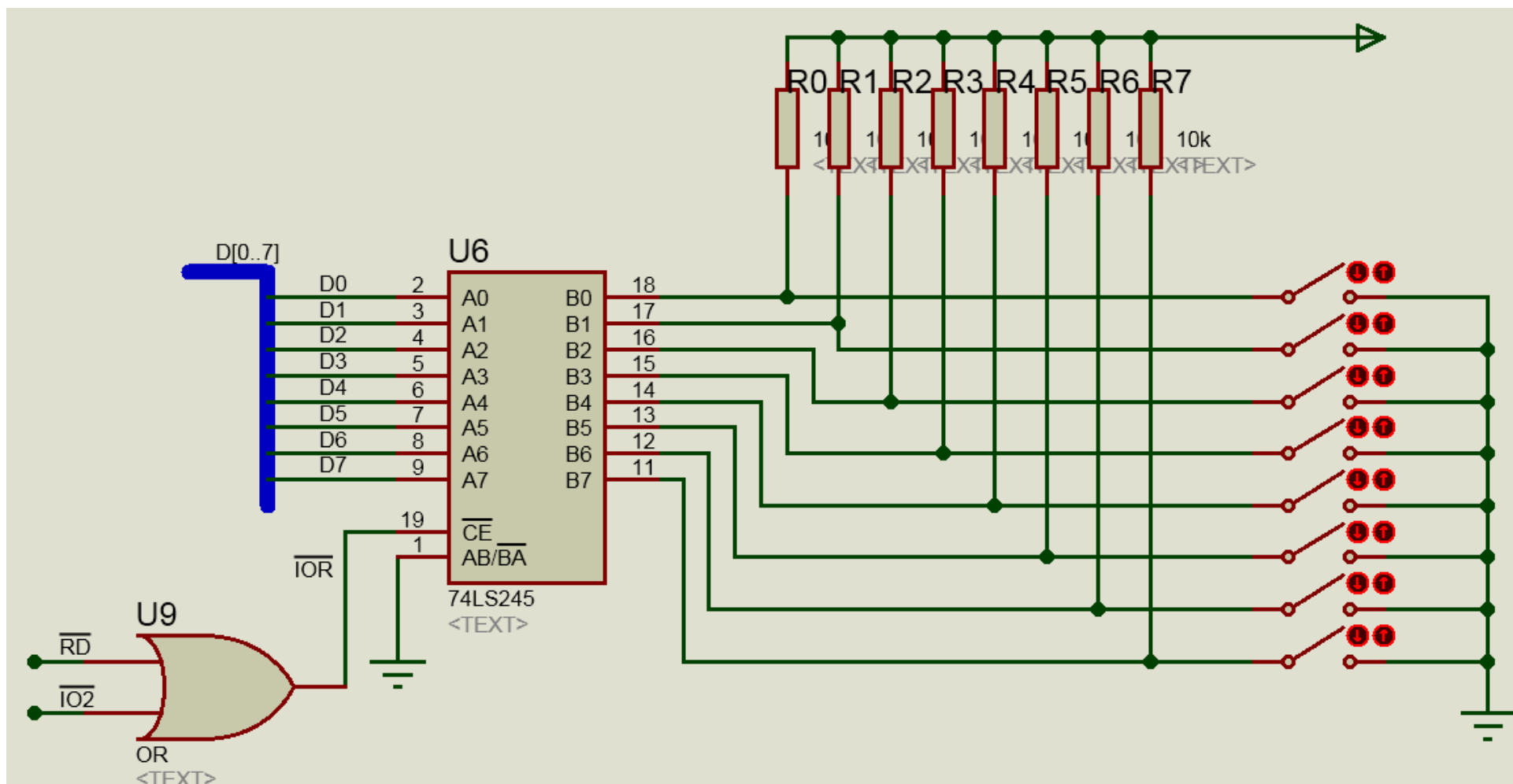
3. 存储器电路（奇偶存储体及控制电路）



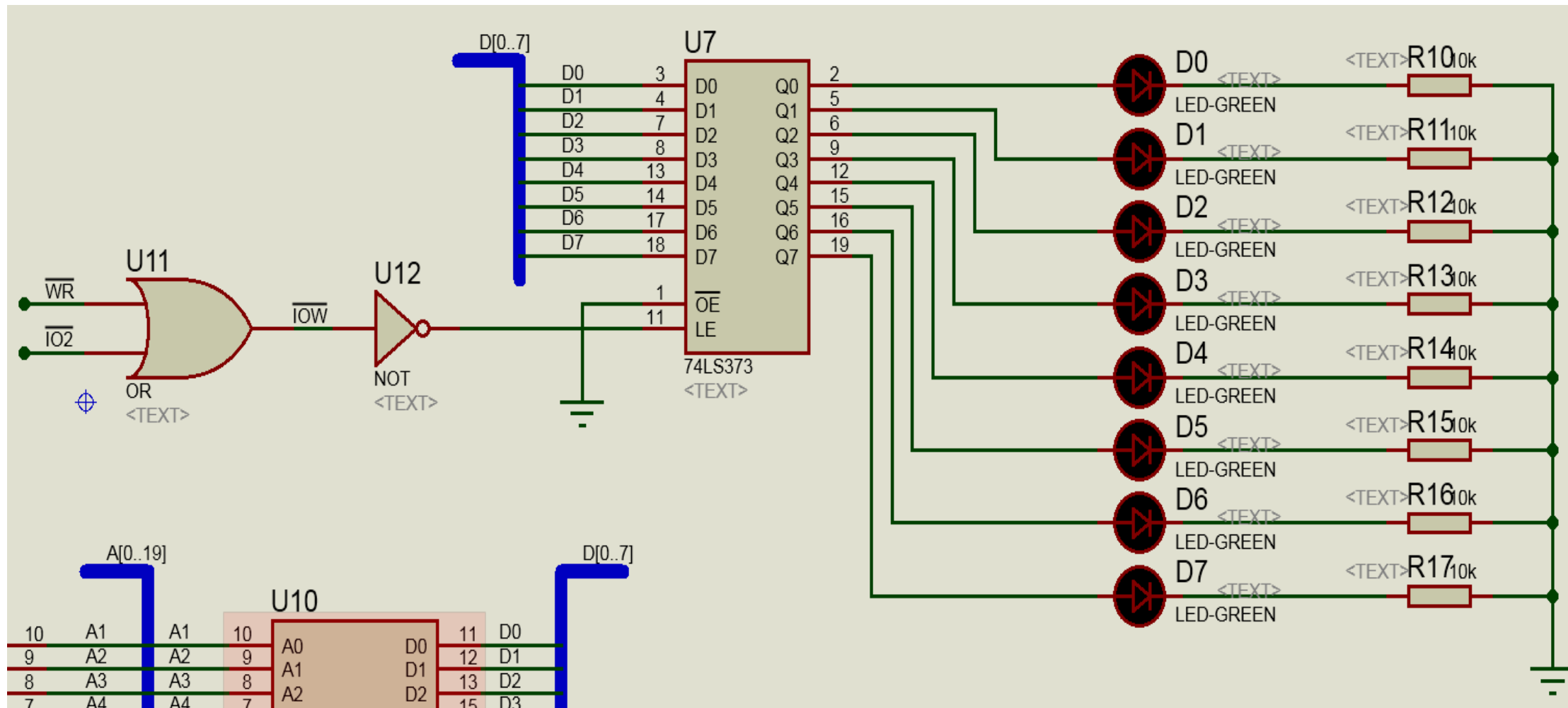
4.IO译码电路



5. 开关信号输入 (IO接口输入缓冲、地址生成)



6.LED灯显示输出（IO接口输出锁存、地址生成）



7. 示波器和逻辑分析仪

The screenshot displays the Proteus 8 Professional Schematic Capture interface. The 'INSTRUMENTS' list on the left includes various tools, with 'OSCILLOSCOPE' and 'LOGIC ANALYSER' highlighted. The main workspace shows a circuit diagram with a logic analyzer component connected to it. The logic analyzer's control panel on the right includes a 'Capture' button and a 'Position' slider. The status bar at the bottom indicates '19 Messages' and 'Root sheet 1'.

1. 按这个按钮 (Click this button)

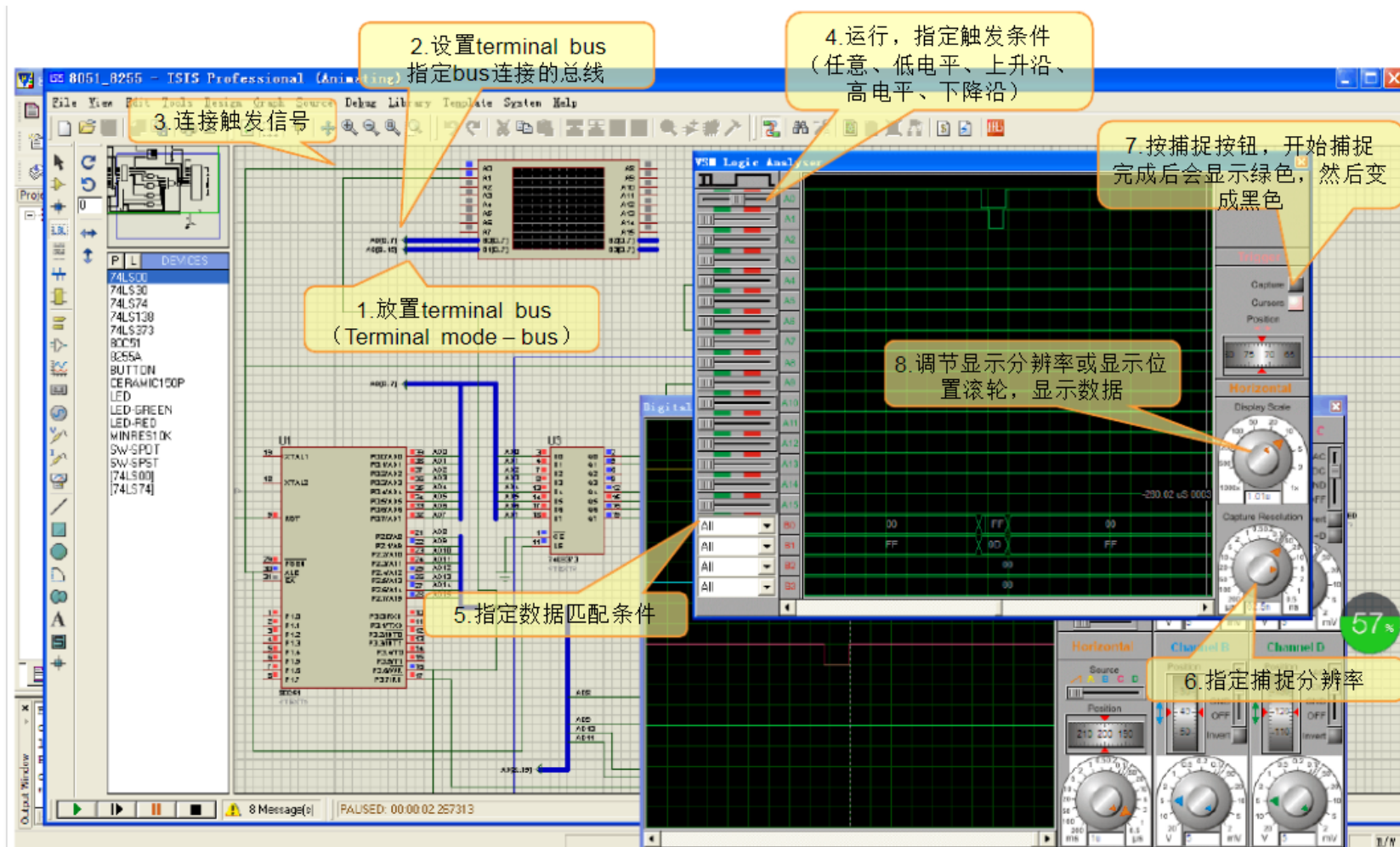
2. 选择示波器 (上) 或逻辑分析仪 (下) (Select oscilloscope (top) or logic analyzer (bottom))

推荐使用逻辑分析仪 (Recommend using logic analyzer)

3. 连线并标明信号名称 (Connect and label signal name)

4. 点击逻辑分析仪上的Capture按钮, 捕捉信号 (Click the Capture button on the logic analyzer, capture signal)

7. 示波器和逻辑分析仪



逻辑分析仪使用注意事项:

1. 每次仿真运行, 都要点击上图标号7的Capture按钮, 此时按钮变成粉红。然后要等到变绿、再变灰之后才会出现波形。
2. 上图标号6的地方, 如果捕捉分辨率过低(即捕捉时间间隔太长), 可能导致波形不能显示。
3. 上图标号4、5的地方可以不用管, 使用默认配置。

使用逻辑分析仪查看波形

New Project - Proteus 8 Professional - Schematic Capture

File Edit View Tool Design Graph Debug Library Template System Help

Base Design

Schematic Capture Source Code

INSTRUMENTS

- OSCILLOSCOPE
- LOGIC ANALYSER
- COUNTER TIMER
- VIRTUAL TERMINAL
- SPI DEBUGGER
- I2C DEBUGGER
- SIGNAL GENERATOR
- PATTERN GENERATOR
- DC VOLTMETER
- DC AMMETER
- AC VOLTMETER
- AC AMMETER
- WATTMETER

2、选中逻辑分析仪

点击按钮

U1: 8086, LOAD_SEG=0x0800

U2: 74273

U3: VSM Logic Analyser

U6: VSM Logic Analyser

U8: NOT

U10: OR

U15: VSM Logic Analyser

4、点击捕获按钮

3、连线

VSM Logic Analyser

Horizontal

Display Scale

Capture Resolution

Trigger

Capture

Cursors

Position

30 25 20

0.5m

1x

1000ns

100ns

10ns

1µs

100µs

1ms

1s

10s

1min

1h

1d

1w

1m

1y

1000ns

100ns

10ns

1µs

100µs

1ms

1s

10s

1min

1h

1d

1w

1m

1y

Proteus中的数据查看功能

1. 点击Debug菜单

The screenshot shows the Proteus 8 Professional interface. The 'Debug' menu is open, displaying various simulation and debugging options. The '8086 Source Code - U1' window shows assembly code. The '8086 Registers - U1' and 'Memory Contents - U10' windows are visible at the bottom. The status bar at the bottom shows 'PAUSED: 0.055978098s'.

2. 此处可以选择需要显示的内容，比如CPU的寄存器、存储器中的数据、逻辑分析仪的波形图等等

单步执行（左）和暂停按钮（右）

1. Simulation Log
2. Watch Window
3. 8086
4. Digital Oscilloscope
5. Memory Contents - U8
6. Memory Contents - U10
7. VSM Logic Analyser

1. Memory Dump - U1
2. Registers - U1
3. Source Code - U1
4. Variables - U1

元器件列表

元件名称	所属类	所属子类	功能说明
8086	Microprocessor ICs	i86 Family	微处理器
74LS373	TTL 74 series	Flip-Flops & Latches	三态输出的8路 锁存器
74LS138	TTL 74LS series	Decoders	3线—8线 译码器
74LS245	TTL 74LS series	Transceivers	8路同相三态 双向总线收发器
62256	Memory ICs	Static RAM	存储器
LED-GREEN	Optoelectics	LEDs	绿色发光二极管
NOT	Simulator Primitives	Gates	非门
OR	Simulator Primitives	Gates	2输入或门
OR4	Modelling Primitives	Digital(Buffers&Gates)	4输入或门
OR8	Modelling Primitives	Digital(Buffers&Gates)	8输入或门
RES	Resistors		电阻
SWITCH	Switchs &Relays	Switchs	开关

汇编源程序

复制下面的汇编程序并替换
Proteus项目main.asm中原有的
代码（用于实现电路原理图1、
2、3部分的系统仿真）

**思考：奇偶存储体的起始物理
地址是多少？理解地址生成电
路的作用**

```
CODE SEGMENT
```

```
    ASSUME CS:CODE
```

```
START: MOV AX,1000H
```

```
        MOV DS,AX
```

```
        MOV SI, 0
```

```
SIM:    MOV BYTE PTR [SI], 01H
```

```
        MOV BYTE PTR [SI+1], 02H
```

```
        MOV BYTE PTR [SI+2], 03H
```

```
        MOV BYTE PTR [SI+3], 04H
```

```
        JMP SIM
```

```
CODE ENDS
```

```
END START
```

汇编源程序

复制下面的汇编程序并替换Proteus项目main.asm中原有的代码
(针对电路原理图4、5、6部分的系统仿真)

```
CODE SEGMENT
    ASSUME CS:CODE
START:
    MOV DX, 200H
    IN AL, DX
    NOT AL
    OUT DX, AL
    JMP START

CODE ENDS
END START
```

汇编源程序

综合前面两种代码，替换Proteus项目main.asm中原有的代码（用于实现电路原理图1~6部分的系统仿真）

```
CODE SEGMENT
ASSUME CS:CODE
START:  MOV AX,1000H
        MOV DS,AX
        MOV SI, 0

        ;存储器操作（向内存中存入数据）
SIM:    MOV BYTE PTR [SI], 01H
        MOV BYTE PTR [SI+1], 02H
        MOV BYTE PTR [SI+2], 03H
        MOV BYTE PTR [SI+3], 04H

        ;输入输出操作（检测开关状态，LED灯显示）
        MOV DX, 200H
        IN  AL, DX
        NOT AL
        OUT DX, AL

        ;跳转到SIM处，重复执行前面的操作
        JMP SIM
CODE ENDS
END START
```

汇编源程序内容扩展（加分）

- 增加8255A、8253、8259等芯片，设计并开发芯片综合应用
- 编写相应的汇编程序代码
- 学期末提交综合应用project报告及Proteus工程文件

例如（不限于）：

电子时钟

- 要求：

- (1) 实现电子时钟的设计，在数码管上显示时、分、秒，显示方式如20：00：00。
- (2) 计时范围可是12小时或24小时。12小时计时从00：00：00到11：59：59，24小时计时从00：00：00到23：59：59。
- (3) 对电子时钟可以进行时、分、秒校正。
- (4) 对时钟显示可以复位。

例如（不限于）：

交通灯控制系统

要求：

- (1) 在十字路口的东西向、南北向分别装有红、绿、黄指示灯。设计其控制电路，控制所有交通灯的亮灭。
- (2) 设计红灯亮20s，绿灯亮17s，黄灯亮3s。当绿灯亮17s后，黄灯再亮3s，作为警示，当黄灯倒计时结束后，红灯亮20s。
- (3) 用发光二极管显示交通灯的亮灭，用数码管的两位显示倒计时的时间。

例如（不限于）：

波形发生器

要求：

- (1) 设计一个波形发生器，能输出三角波、矩形波、锯齿波、正弦波等波形信号。
- (2) 设计一个 4×4 的矩阵键盘，对输出的波形类型和幅值等进行控制。当键盘输入A~D时，选择输出波形的类型；当键盘输入1~5时，控制输出波形的幅值（五档）。
- (3) 示波器显示输出波形。